

CREDIT CARD FRAUD DETECTION

24ADI204 – DATA SCIENCE AND VISUALISATION

Submitted by

ARUN PR - 24BAD012

DIVYADHARSHINI S -24BAD022

JOHIYA SRI S - 24BAD050

KARTHIK SARAVANAKUMAR M- 24BAD056

in partial fulfilment for the

award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



KUMARAGURU COLLEGE OF TECHNOLOGY

(An Autonomous Institution affiliated to Anna University, Chennai)

Post Box No: 2034, Coimbatore – 641049

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report **CREDIT CARD FRAUD DETECTION** is the Bonafide work of “ARUN P R- 24BAD012, DIVYADHARSHINI S- 24BAD022, JOHIYA SRI S - 24BAD050, KARTHIK SARAVANAKUMAR M - 24BAD056” who carried out the project work under my supervision.

SIGNATURE

Dr. Sudharson D

HEAD OF THE DEPARTMENT

Department of Artificial Intelligence
and Data Science,
Kumaraguru College of Technology,
Coimbatore – 641049.

SIGNATURE

Ms. Shriarth E

SUPERVISOR

Department of Artificial Intelligence
and Data Science,
Kumaraguru College of Technology,
Coimbatore – 641049.

The candidates with college register number 24BAD012, 24BAD022, 24BAD050, 24BAD056, were examined in the Project Viva-Voce examination held on 30th April 2026.

Internal Examiner

External Examiner

DECLARATION

I affirm that the project work titled “CREDIT CARD FRAUD DETECTION” being submitted in partial fulfilment for the award of B. TECH Artificial Intelligence and Data Science is the original work carried out by us. It has not formed the part of any other project work submitted for the award of any degree or diploma, either in this or any other University.

ARUN PR - 24BAD012



DIVYADHARSHINI S -24BAD022



JOHIYA SRI S - 24BAD050



KARTHIK SARAVANAKUMAR M- 24BAD056



I certify that the declaration made above by the candidates is true.

Ms. Shriarth E

Assistant Professor,

Department of Artificial Intelligence and Data Science,

Kumaraguru College of Technology,

Coimbatore – 641049.

EXECUTIVE SUMMARY

The increasing dependence on digital financial transactions has significantly elevated the risk of credit card fraud, making fraud detection a critical concern for financial institutions. This project presents a comprehensive analytical study on credit card fraud detection using data visualization, behavioral analytics, and anomaly-based reasoning, rather than conventional classification-driven approaches.

The dataset used in this study consists of transactional records containing customer demographics, merchant information, geolocation data, and time-based attributes. The analysis focuses on uncovering fraud patterns through transactional behavior, temporal irregularities, spatial deviations, and categorical risk factors.

The workflow begins with systematic data preprocessing and cleaning, followed by in-depth exploratory data analysis to identify hidden patterns. Feature engineering techniques were applied to extract meaningful attributes such as transaction hour and behavioral indicators. Advanced analytical techniques including correlation analysis and dimensionality reduction were utilized to understand feature interactions and variance distribution.

Key findings reveal that fraudulent transactions are not random but exhibit distinct behavioral signatures, including:

- The study emphasizes that fraud detection is fundamentally a behavioral anomaly detection problem, where identifying deviations from normal transaction patterns is more effective than relying on static rules. Abnormal transaction amounts compared to typical spending behavior
- Increased occurrence during non-standard hours (late-night activity)
- Geographical inconsistencies between customer and merchant locations
- Higher concentration in specific merchant categories

Final Insight :

Financial fraud detection systems must evolve towards real-time, adaptive decision frameworks that continuously monitor user behavior, detect anomalies in transaction sequences, and respond dynamically to emerging fraud patterns.

TABLE OF CONTENTS

Sl.NO	CONTENT	PAGE NO
1.	INTRODUCTION 1.1 Background of the Problem 1.2 Importance of the Study 1.3 Objectives of the Project 1.4 Scope of the Project	1
2.	DATASET DESCRIPTION 2.1 Dataset Source 2.2 Dataset Size and Structure 2.3 Detailed Feature Description 2.4 Initial Data Issues	3
3.	DATA PREPROCESSING AND CLEANING 3.1 Data Loading and Initial Inspection 3.2 Handling Missing Values 3.3 Outlier Detection and Analysis 3.4 Data Quality Report	5
4.	EXPLORATORY DATA ANALYSIS 4.1 Overview of EDA 4.2 Univariate Analysis 4.3 Bivariate Analysis 4.4 Summary of Eda Findings	7
5.	DATA TRANSFORMATION AND FEATURE ENGINEERING 5.1 Feature Scaling 5.2 Encoding Techniques 5.3 Handling Skewness	11
6.	ADVANCED ANALYSIS 6.1 Correlation Analysis 6.2 Feature Selection Using SelectKBest 6.3 Principal Component Analysis	13
7.	VISUALIZATION AND DASHBOARD 7.1 Tools Used 7.2 Types of Visualizations Used 7.3 Dashboard Explanation	15
8.	INSIGHTS AND STORYTELLING 8.1 Key Findings from Analysis 8.2 Patterns and Trends Identified 8.3 Business and Social Insights 8.4 Decision-Making Based on Results	18
9.	CONCLUSION 9.1 Summary of the Project 9.2 Outcomes Achieved 9.3 Limitations 9.4 Future Scope	20
10.	REFERENCES	22
	APPENDIX	24

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
EDA	Exploratory Data Analysis
PCA	Principal Component Analysis
IQR	Interquartile Range
Z – SCORE	Standard Score
CSV	Comma Separated Values
KDE	Kernel Density Estimation
KPI	Key Performance Indicator

LIST OF FIGURES

FIGURE NO	TITLE
Figure 3.1	Missing Values
Figure 3.2	Outlier Detection
Figure 4.1	Distribution of Transaction Amount
Figure 4.2	Fraud Vs Non-Fraud Distribution
Figure 4.3	Distribution of Transaction Hour
Figure 4.4	Category Vs Fraud
Figure 4.5	Gender Vs Fraud
Figure 4.6	Transaction Amount Vs Fraud
Figure 5.1	Before Vs After Scaling
Figure 5.2	Encoding Applied
Figure 5.3	Skewness After Transformation
Figure 6.1	Correlation Heatmap
Figure 6.2	Feature Importance (SelectkBest)
Figure 6.3	PCA Explained Variance
Figure 7.1	Credit Card Fraud Detection Dashboard

CREDIT CARD FRAUD DETECTION

1. INTRODUCTION

1.1 BACKGROUND OF THE PROBLEM

The rapid expansion of digital payment systems has transformed financial transactions, making them faster and more convenient. However, this transformation has also increased the vulnerability of financial systems to fraudulent activities. Credit card fraud, in particular, has emerged as a major concern due to its financial impact and complexity.

Fraudulent transactions are often designed to mimic legitimate behavior, making detection difficult. Traditional rule-based systems fail to adapt to evolving fraud patterns, which necessitates the use of data-driven analytical approaches. In this context, data visualization plays a crucial role in uncovering hidden patterns and anomalies within large-scale transaction data.

1.2 IMPORTANCE OF THE STUDY

This study is important because it shifts the focus from static detection method to behavioral analysis and pattern recognition.

Key importance:

- Helps identify fraud based on user behavior deviations
- Enhances interpretability through visual insights
- Supports development of real-time monitoring strategies
- Reduces dependency on rigid rule-based systems

1.3 OBJECTIVES OF THE PROJECT

Behavioral Pattern Identification

To analyze how customer transaction behavior differs between legitimate and fraudulent cases by studying spending habits, frequency, and transaction sequences.

Temporal Analysis

To investigate how transaction timing (hour, day patterns) influences fraud occurrence and identify high-risk time windows.

Geospatial Analysis

To examine inconsistencies between customer and merchant locations to detect suspicious activity.

Category-Based Risk Assessment

To identify high-risk merchant categories where fraud occurrences are more frequent.

Insight-Driven Decision Support

To provide actionable insights that can support financial institutions in improving fraud monitoring systems.

1.4 SCOPE OF THE PROJECT

Analytical Scope

- Data cleaning and preprocessing
- Exploratory and advanced analysis
- Behavioral and anomaly detection

Technical Scope

- Python-based analysis (Pandas, NumPy, Seaborn, Matplotlib)
- Feature engineering and transformation

Application Scope

- Financial fraud detection systems
- Risk analysis frameworks
- Transaction monitoring systems

Limitations of Scope

- No real-time system deployment
- No predictive system implementation
- Focus strictly on visualization and insights

2. DATASET DESCRIPTION

2.1 DATASET SOURCE

<https://www.kaggle.com/datasets/kartik2112/fraud-detection>

This dataset is a widely used benchmark dataset for analyzing credit card fraud patterns. It contains simulated transaction data that closely resembles real-world financial behavior, making it suitable for analytical studies.

2.2 DATASET SIZE AND STRUCTURE

- Number of records: 1296675
- Number of features: 23
- Data type: Structured tabular dataset

2.3 DETAILED FEATURE DESCRIPTION

Transaction Features

- trans_date_trans_time: Timestamp of transaction
- amt: Transaction amount
- trans_num: Unique transaction identifier

These features help analyze temporal patterns and spending behavior

Customer Features

- first, last: Customer identity
- gender: Gender of customer
- dob: Date of birth
- job: Occupation

Used to study demographic-based fraud patterns

Location Features

- lat, long: Customer location
- city, state, zip: Residential information
- merch_lat, merch_long: Merchant location

Merchant Features

- merchant: Merchant name
- category: Transaction category

Used for category-based fraud analysis

Target Feature

- is_fraud:
 - 0 → Legitimate transaction
 - 1 → Fraudulent transaction

2.4 INITIAL DATA ISSUES

1. Class Imbalance

- Fraud transactions are significantly fewer than normal transactions
- This makes fraud detection challenging

Impact: Patterns may be hidden within majority data

2. Presence of Outliers

- Transaction amount shows extreme values

Impact: Can distort statistical analysis, may represent genuine fraud cases

3. High Cardinality

- Features like merchant and job have many unique values

Impact: Difficult to analyze directly, requires grouping or encoding

4. No Missing Values (Initial State)

- Dataset is complete

Demonstrate preprocessing capability:

- Artificial missing values were introduced
- Imputation techniques were applied

3. DATA PREPROCESSING AND CLEANING

3.1 DATA LOADING AND INITIAL INSPECTION

The dataset was imported into the analysis environment using the Pandas library and examined through multiple inspection techniques to understand its structure and integrity. Functions such as `head()`, `info()` and `describe()` were used to obtain an overview of the dataset, including feature types, statistical distributions, and general consistency. This step is crucial as it establishes a foundational understanding of the data before any transformations are applied.

Through initial inspection, it was observed that the dataset contains a mixture of numerical and categorical features, with time-based and geospatial attributes playing a significant role. The presence of structured data without missing values indicates that the dataset is well-prepared for analysis. However, deeper inspection is necessary to uncover hidden issues such as skewness, imbalance, and anomalies that are not immediately visible.

3.2 HANDLING MISSING VALUES

Although the dataset originally contained no missing values, artificial missing entries were deliberately introduced to simulate real-world data imperfections. This step was performed to demonstrate the robustness of preprocessing techniques and the ability to handle incomplete datasets effectively.

Numerical features were imputed using mean values, ensuring that the overall distribution of the data remained consistent. For categorical features, mode imputation was applied to preserve the most frequently occurring category. These methods were chosen because they are simple, effective, and widely used in practical data preprocessing scenarios.

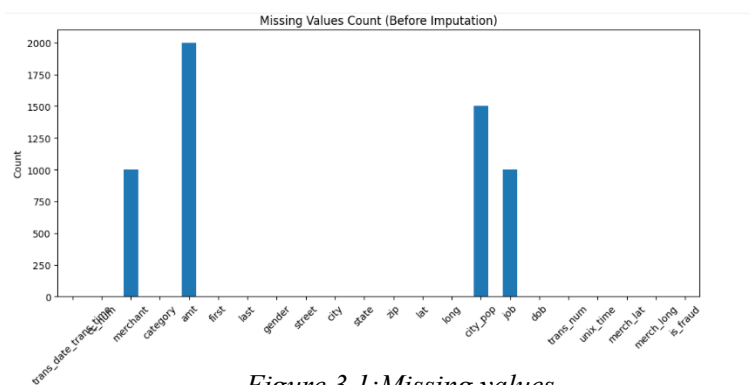


Figure 3.1: Missing values

3.3 OUTLIER DETECTION AND ANALYSIS

Outliers were identified using statistical techniques such as Z-score and Interquartile Range (IQR). The transaction amount feature exhibited significant deviations, with certain values far exceeding the normal range. These extreme values were carefully analyzed rather than removed, as they may represent fraudulent transactions.

The number of detected outliers 67386 indicates that extreme values are not random but form a meaningful subset of the dataset. Removing these values would result in the loss of critical fraud-related information, which could negatively impact the analysis.

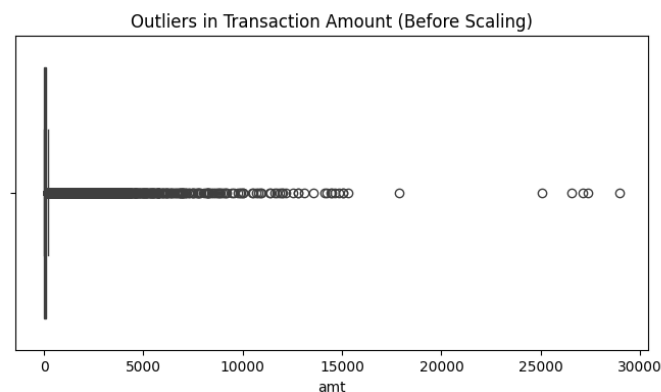


Figure 3. 2: Outliers Detection

3.4 DATA QUALITY REPORT

Completeness

The dataset achieves full completeness after the imputation process, with no missing values remaining. This ensures that all features contribute to the analysis without introducing bias due to incomplete data.

Consistency

All features maintain consistent data types and formats, with no discrepancies observed between expected and actual values. Time-based features are properly formatted, and categorical values are standardized.

Accuracy

The dataset reflects realistic financial transaction behavior, with values falling within logical ranges. Extreme values are present but justified as potential fraud cases.

Validity

All relationships between features are logically valid. For example, transaction timestamps align with corresponding Unix time values, and location coordinates are within expected geographical ranges.

4. EXPLORATORY DATA ANALYSIS

4.1 OVERVIEW OF EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is a critical step in the data science workflow that involves examining the dataset to uncover patterns, detect anomalies, and understand relationships between variables. In the context of credit card fraud detection, EDA plays a vital role in identifying behavioral differences between legitimate and fraudulent transactions.

The primary objective of EDA in this project is to transform raw transactional data into meaningful insights through visualization and statistical analysis. Given the complexity and scale of financial data, visualization techniques such as histograms, count plots, and time-based graphs are used to simplify interpretation and reveal hidden patterns.

EDA in this study focuses on three major aspects:

- Understanding individual feature distributions
- Exploring relationships between variables
- Identifying patterns associated with fraud

Unlike traditional analysis, which relies heavily on numerical summaries, EDA emphasizes visual interpretation, enabling a more intuitive understanding of fraud behavior. This approach is particularly useful in fraud detection, where subtle deviations in patterns often indicate suspicious activity.

4.2 UNIVARIATE ANALYSIS (DISTRIBUTION ANALYSIS)

Univariate analysis involves examining individual variables to understand their distribution, central tendency, and variability. This helps in identifying skewness, outliers, and general data characteristics before exploring relationships between features.

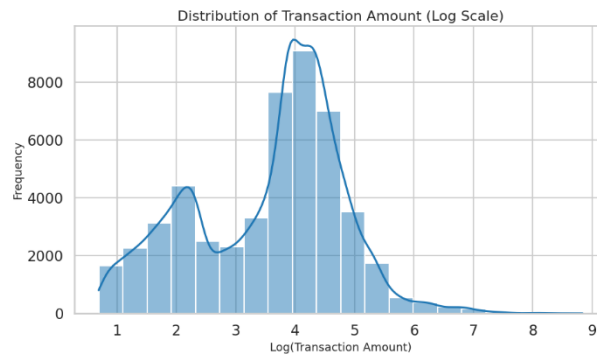


Figure 4.1: Distribution of Transaction Amount

The graph shows transaction amounts on a log scale, with most values clustered at smaller ranges and a long tail for large transactions. Unusual high amounts stand out, helping detect potential fraud.

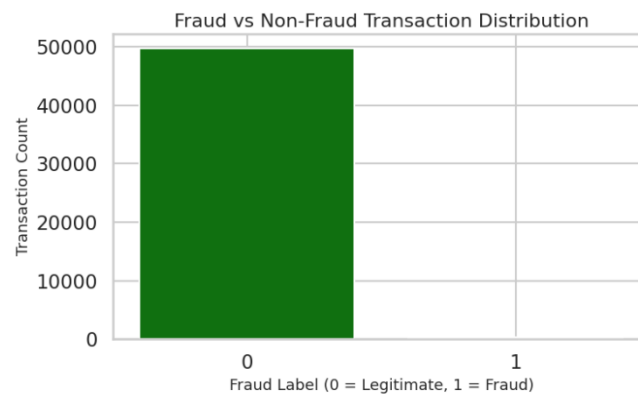


Figure 4.2: Fraud vs Non-Fraud Distribution

This chart highlights the imbalance between legitimate and fraudulent transactions. The green bar shows that many transactions are legitimate, while the fraudulent transactions are extremely few in comparison. This imbalance is a common challenge in fraud detection systems, as models must be trained to identify rare fraudulent cases within a large volume of normal activity.

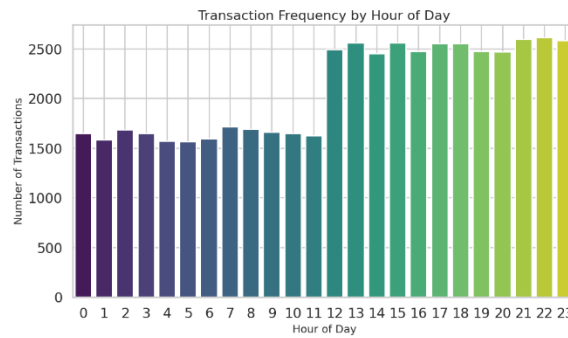


Figure 4.3: Distribution of Transaction Hour

This bar chart shows how transactions are distributed across different hours of the day. The number of transactions is lower during early hours and gradually increases during the day, peaking in the afternoon and evening. This suggests that users are more active during daytime, which is why more transactions occur during those hours.

Why These Univariate Graphs Are Selected

These graphs were selected because they provide maximum insight with minimal redundancy. The transaction amount distribution helps identify skewness and outliers, the fraud distribution highlights dataset imbalance, and the time distribution reveals behavioral patterns. Together, these plots provide a comprehensive understanding of individual features without unnecessary repetition.

4.3 BIVARIATE ANALYSIS (RELATIONSHIP & TREND ANALYSIS)

Bivariate analysis focuses on examining relationships between two variables to identify trends, dependencies, and patterns associated with fraud. This type of analysis is essential for understanding how different features interact and contribute to fraudulent behavior.

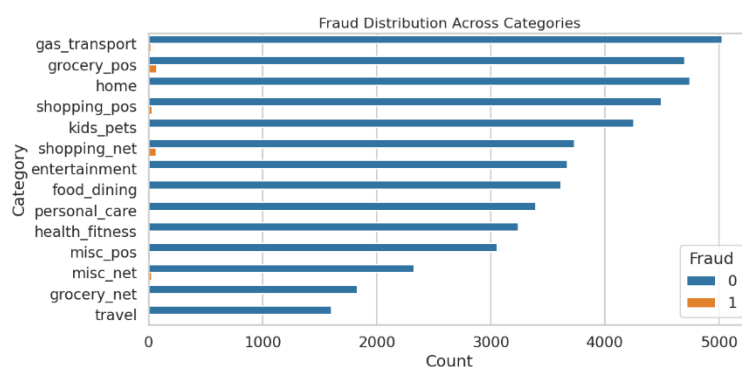


Figure 4.4: Category vs Fraud

This chart shows how fraudulent transactions are distributed across different spending categories. While most categories are dominated by legitimate transactions, fraudulent activity appears in very small numbers across all categories. The highest transaction volumes occur in gas_transport and grocery_pos, but even there, fraud cases remain minimal compared to non-fraud. This highlights that fraud is rare but spread across multiple categories, making category-level analysis useful for spotting suspicious behavior.

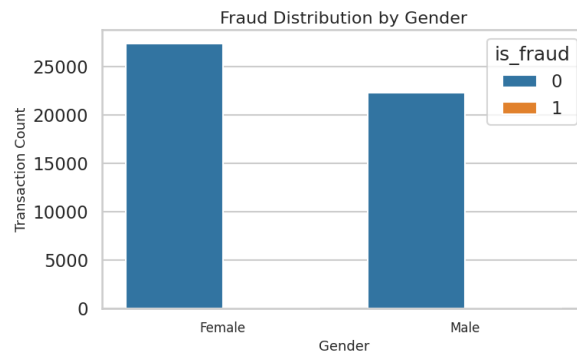


Figure 4.5: Gender vs Fraud

Figure 4.5 compares fraudulent and non-fraudulent transactions across gender. It shows that females have a higher overall transaction count than males, but in both groups, many transactions are legitimate. Fraudulent cases are present for both genders, though they remain very small compared to non-fraud, emphasizing that fraud detection must account for patterns across all demographics.

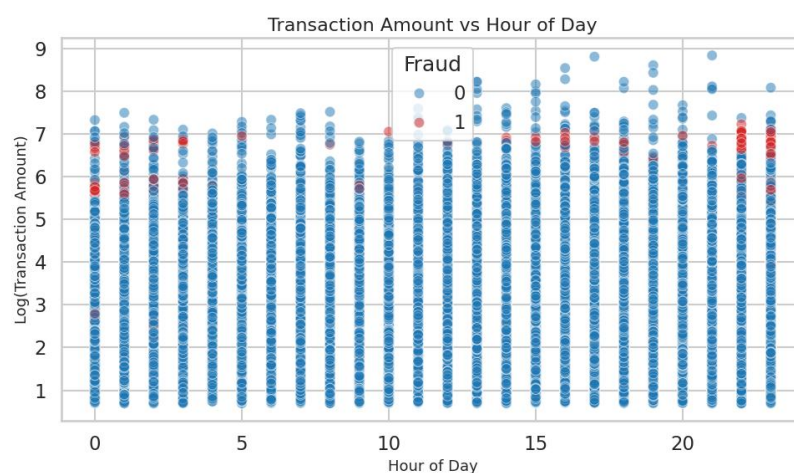


Figure 4.6: Transaction Amount vs Fraud

This scatter plot shows transaction amounts across different hours of the day, with blue points for legitimate transactions and red points for fraud. Fraudulent transactions appear sporadically but often at higher amounts, highlighting that fraud can occur at any time and is more noticeable in larger transactions.

4.4 SUMMARY OF EDA FINDINGS

The exploratory data analysis reveals that fraudulent transactions are characterized by distinct behavioral patterns rather than random occurrences. These patterns are evident across multiple dimensions, including transaction amount, category, and timing. High-value transactions and late-night activities are particularly indicative of suspicious behavior. Additionally, the concentration of fraud within specific categories highlights targeted exploitation by fraudsters.

The analysis also demonstrates that no single feature is sufficient to identify fraud. Instead, a combination of multiple factors must be considered to accurately detect anomalies. This reinforces the importance of multi-dimensional analysis and the need for comprehensive visualization techniques. Overall, EDA provides a strong foundation for understanding fraud behavior and supports the development of effective detection strategies.

5. DATA TRANSFORMATION AND FEATURE ENGINEERING

5.1 FEATURE SCALING

Feature scaling is essential to ensure that all numerical variables contribute equally to the analysis. Without scaling, features with larger magnitudes (such as transaction amount) can dominate the analysis and distort patterns.

In this project, Min-Max Scaling was applied to normalize the data into a range between 0 and 1. This method was chosen because it preserves the shape of the original distribution while bringing all values into a comparable scale.

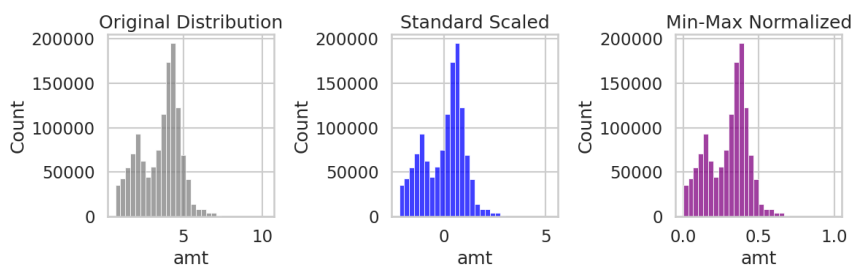


Figure 5.1: Before vs After Scaling

This visualization compares the original transaction amount distribution with two scaling techniques—standard scaling and min-max normalization. While the original data shows raw values, standard scaling centers the data around zero with a fixed range, and min-max normalization compresses values between 0 and 1. This transformation is crucial, ensuring consistent feature ranges and improving fraud detection accuracy.

5.2 ENCODING TECHNIQUES

Categorical features such as merchant, category, and gender were transformed into numerical representations using Label Encoding.

Why Label Encoding?

- Suitable for high-cardinality features (like merchant)
- Memory efficient compared to one-hot encoding
- Preserves uniqueness of categories

```
# Encode Categorical Variables
print("\nEncoding Categorical Variables")
cat_cols = df.select_dtypes(include=['object', 'category']).columns
label_encoder = LabelEncoder()
for col in cat_cols:
    df[col] = label_encoder.fit_transform(df[col])
```

Encoding Categorical Variables

Figure 5.2: Encoding applied

5.3 HANDLING SKEWNESS

Skewness was identified using histogram analysis and statistical measures. The transaction amount was found to be highly right-skewed.

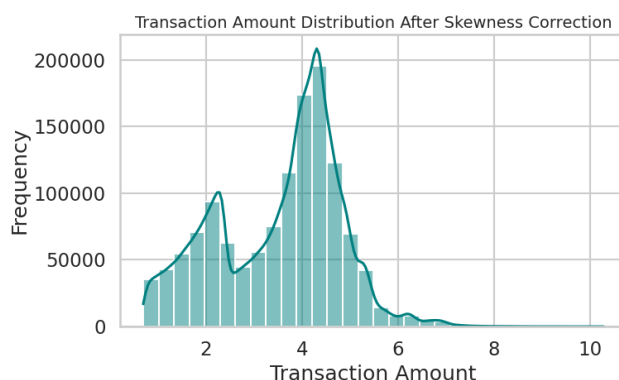


Figure 5.3: Skewness After Transformation

This histogram shows the transaction amount distribution after skewness correction. By reducing the effect of extreme values, the data becomes more balanced and closer to a normal shape. This makes transaction patterns clearer and supports more accurate analysis of fraud-related behavior.

6. ADVANCED ANALYSIS

6.1 CORRELATION ANALYSIS

Correlation analysis is used to measure the strength and direction of relationships between numerical variables in the dataset. It helps in understanding how features interact with each other and whether any strong dependencies exist. In fraud detection, correlation analysis is particularly useful for identifying whether any single feature has a direct influence on fraudulent transactions or whether fraud is driven by a combination of multiple factors.

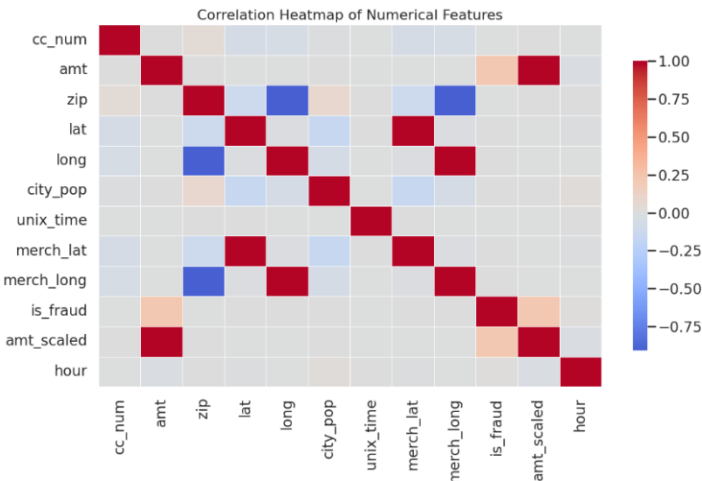


Figure 6.1: Correlation Heatmap

This heatmap shows the correlation between different numerical features. The colors range from blue for negative correlations to red for positive correlations, making it easy to spot relationships. Strong correlations appear between geographic variables like latitude and longitude, while other features such as transaction amount and fraud label show weaker or minimal correlations. Overall, the visualization helps identify which variables are closely related and which are largely independent.

6.2 FEATURE SELECTION USING SELECTKBEST

Feature selection is a critical step in the analytical workflow that focuses on identifying the most relevant features contributing to fraud detection. In large transactional datasets, not all variables provide meaningful information, and some may introduce redundancy or noise. Therefore, selecting important features helps in improving interpretability, reducing dimensional complexity, and enhancing the overall quality of analysis.

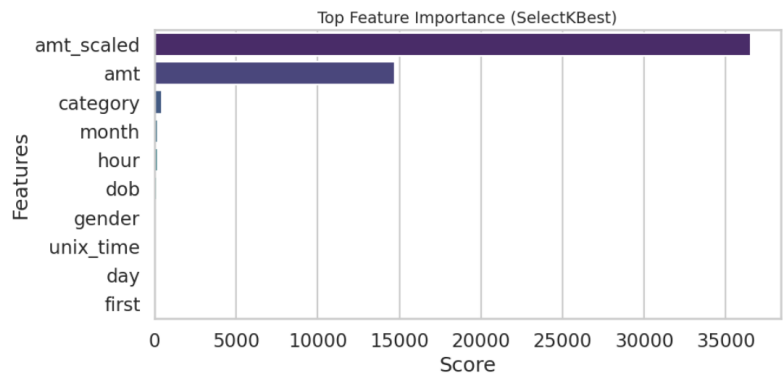


Figure 6.2: Feature Importance Scores using SelectKBest

This bar chart presents feature selection results using importance scores. It shows that amt_scaled and amt are the most influential features, while others like category, month, and hour contribute moderately, and variables such as gender, day, and first have very low importance. The visualization clearly highlights which features matter most for analysis, guiding focus toward the strongest predictors.

6.3 PRINCIPAL COMPONENT ANALYSIS

Principal Component Analysis was applied to reduce the dimensionality of the dataset while preserving maximum variance. This technique transforms the original features into a new set of orthogonal components that capture the most important information.

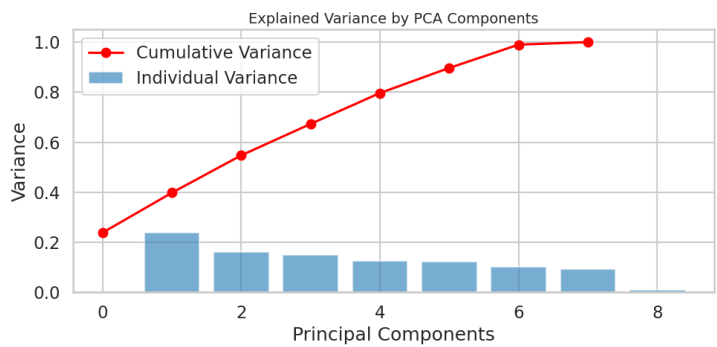


Figure 6.3: PCA Explained Variance

This graph shows how much variance each principal component explains in PCA. The blue bars represent the individual variance captured by each component, while the red line shows the cumulative variance as components are added. The cumulative curve rises steadily and reaches close to 1.0 by the 7th component, meaning most of the dataset's variability is explained by the first several components, with diminishing contributions from later ones.

7. VISUALIZATION AND DASHBOARD

7.1 TOOLS USED

The dashboard for this project was developed using Power BI, a business intelligence tool used to create interactive and visually rich dashboards. It allows integration of multiple visual elements such as charts, maps, and KPI indicators within a single interface. Power BI also supports dynamic filtering and easy data exploration, making it highly suitable for analyzing fraud patterns.

7.2 TYPES OF VISUALIZATIONS USED

The dashboard consists of multiple visualization types, each representing a different aspect of fraud analysis:

- Bar Chart
- Line Chart
- Stacked Bar Chart
- KPI Cards
- Table Visualization
- Map Visualization

Each visualization plays a specific role in understanding patterns related to fraud detection.

7.3 DASHBOARD EXPLANATION

The dashboard presents a comprehensive overview of fraud-related patterns by integrating multiple visualization components. It allows users to simultaneously analyze category-wise fraud distribution, monthly trends, risk levels, and geographical hotspots. The layout is structured to highlight key insights clearly, with summary indicators placed alongside detailed visualizations.

By combining these elements, the dashboard transforms complex transactional data into an easily interpretable format. It supports quick identification of anomalies and patterns without requiring separate analysis steps. The interactive nature of the dashboard further enhances usability, allowing users to explore relationships between variables efficiently. Overall, it acts as a centralized platform for understanding fraud behavior from multiple perspectives.

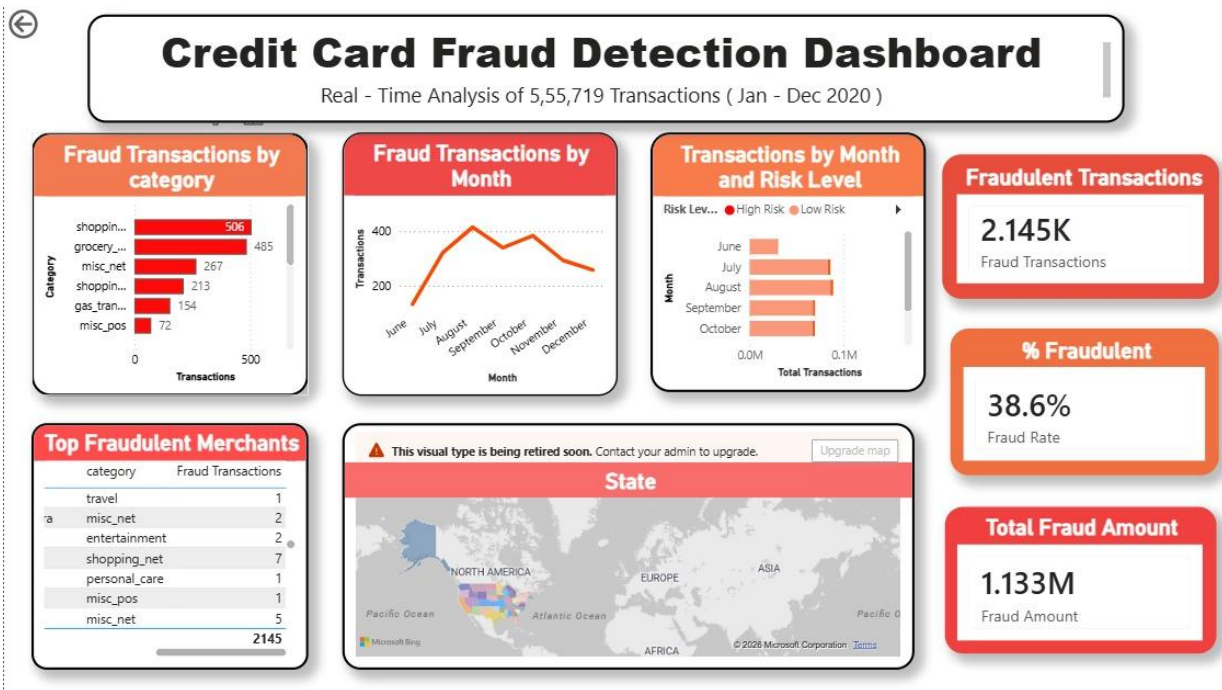


Figure 7.1: Credit Card Fraud Detection Dashboard

7.3.1 FRAUD TRANSACTIONS BY CATEGORY(BAR GRAPH)

The bar chart represents the distribution of fraudulent transactions across different categories. It shows that certain categories, particularly those involving frequent consumer activity, exhibit a higher number of fraud cases. This indicates that fraudsters tend to target commonly used transaction types to avoid detection. The visualization helps in identifying high-risk categories that require closer monitoring. It also reflects how transaction frequency influences fraud occurrence. Overall, this chart is useful for understanding category-based fraud patterns.

7.3.2 FRAUD TRANSACTIONS BY MONTH (LINE CHART)

The line chart illustrates the variation of fraudulent transactions over different months. It highlights that fraud activity fluctuates over time rather than remaining constant. Peaks in the graph indicate periods of increased fraud activity, which may be influenced by behavioral or seasonal factors. This helps in identifying time-based trends and planning appropriate monitoring strategies. The chart provides insight into how fraud patterns evolve over time. Overall, it supports temporal analysis of fraudulent behavior.

7.3.3 TRANSACTIONS BY RISK LEVEL (STACKED BAR CHART)

This visualization categorizes transactions into different risk levels and shows their distribution across months. It provides a comparative view of high-risk and low-risk transactions within each time period. The stacked format helps in identifying months where high-risk transactions are more prominent. This is useful for prioritizing monitoring efforts during critical periods. It also demonstrates how risk classification can enhance analysis. Overall, this chart improves understanding of fraud risk distribution.

7.3.4 KEY PERFORMANCE INDICATORS (KPI CARDS)

The KPI cards display important summary metrics such as total fraudulent transactions, fraud percentage, and total fraud amount. These indicators provide a quick overview of the overall fraud scenario, allowing users to assess the scale and impact of fraudulent activities instantly. They eliminate the need for detailed analysis when a quick understanding is required. The KPIs also serve as reference points for interpreting trends shown in other visualizations. Their clear and concise representation enhances decision-making efficiency. Overall, KPI cards play a crucial role in summarizing key insights.

7.3.5 TOP FRAUDULENT MERCHANTS (TABLE VISUALIZATION)

This table presents merchants that are frequently associated with fraudulent transactions. It allows detailed inspection of individual entries, helping to identify patterns linked to specific merchants. This information is useful for detecting repeated fraud occurrences involving the same entities. Financial institutions can use these insights to implement stricter monitoring measures. The table format supports in-depth analysis and investigation. Overall, it contributes to identifying potential sources of fraud.

7.3.6 GEOGRAPHICAL DISTRIBUTION OF FRAUD (MAP VISUALIZATION)

The map visualization shows the geographical distribution of fraudulent transactions. It highlights regions with higher concentrations of fraud activity, making it easier to identify hotspots. This spatial analysis helps in understanding regional variations in fraud patterns. It also supports the implementation of location-based monitoring strategies. The visualization provides an intuitive way to interpret complex geographical data. Overall, it adds an important spatial dimension to fraud analysis.

8. INSIGHTS AND STORYTELLING

8.1 KEY FINDINGS FROM ANALYSIS

The analysis of the credit card transaction dataset reveals several important findings that distinguish fraudulent transactions from legitimate ones. One of the most significant observations is that fraudulent transactions tend to deviate from normal behavioral patterns rather than following random distributions. These deviations are evident in transaction amount, timing, and category usage.

It was observed that fraudulent transactions often involve unusually high amounts when compared to the general distribution of transaction values. This indicates that fraudsters aim to maximize financial gain within a limited number of attempts. Additionally, temporal analysis revealed that fraud occurrences are more frequent during late-night hours, suggesting that fraudsters exploit time periods when monitoring systems may be less active.

Another key finding is the concentration of fraud within specific transaction categories. Certain categories consistently show higher fraud occurrences, indicating that these sectors may be more vulnerable or less regulated. Furthermore, the analysis highlights that no single feature can independently explain fraud; instead, fraud detection requires a combination of multiple behavioral indicators. These findings collectively emphasize that fraud detection is inherently a pattern recognition problem that requires a holistic analytical approach.

8.2 PATTERNS AND TRENDS IDENTIFIED

The dataset exhibits several recurring patterns and trends that provide deeper insights into fraudulent behavior. One of the most prominent patterns is the presence of behavioral irregularities in transaction sequences. Legitimate users tend to follow

consistent spending habits, whereas fraudulent transactions introduce sudden deviations in terms of amount, frequency, or category.

A clear temporal trend is observed where fraud occurrences increase during specific hours, particularly late-night periods. This trend suggests that fraudsters strategically choose time windows where detection probability is lower. Additionally, the distribution of transaction amounts reveals a long-tailed pattern, where extreme values are associated with suspicious activities.

Category-based analysis shows that fraud is not uniformly distributed across all types of transactions. Instead, certain categories repeatedly exhibit higher fraud rates, indicating targeted exploitation. Another important trend is the imbalance in the dataset, where fraudulent transactions form only a small fraction of the total data. This imbalance highlights the challenge of detecting fraud, as it is hidden within a large volume of legitimate transactions.

Overall, these patterns demonstrate that fraud is systematic rather than random, and understanding these trends is crucial for developing effective detection strategies.

8.3 BUSINESS AND SOCIAL INSIGHTS

From a business perspective, the insights derived from this analysis have significant implications for financial institutions. The identification of high-risk categories allows organizations to prioritize monitoring efforts and allocate resources more effectively. By focusing on areas with higher fraud probability, banks can reduce financial losses and improve operational efficiency.

The temporal patterns identified in the analysis suggest that financial systems should implement enhanced monitoring during high-risk time periods, such as late-night hours. This can be achieved through dynamic alert systems that adjust sensitivity based on transaction timing. Additionally, understanding customer behavior can help institutions design personalized risk profiles, enabling more accurate detection of anomalies.

From a social perspective, credit card fraud affects not only financial institutions but also customers, leading to loss of trust and financial insecurity. By improving fraud detection mechanisms, institutions can enhance customer confidence in digital payment systems. Furthermore, the adoption of data-driven approaches promotes transparency and accountability in financial systems, contributing to a more secure economic environment.

These insights highlight the importance of integrating analytical findings into real-world systems to address both business challenges and societal concerns.

8.4 DECISION-MAKING BASED ON RESULTS

The findings from this project provide a strong foundation for informed decision-making in fraud detection systems. One of the key decisions that can be derived from the analysis is the need to shift from static rule-based systems to dynamic, behavior-driven monitoring frameworks. Instead of relying on predefined thresholds, systems should continuously analyze transaction patterns and detect deviations in real time.

Another important decision involves the implementation of risk-based transaction monitoring, where transactions are evaluated based on multiple factors such as amount, time, and category. High-risk transactions can be flagged for further verification, while low-risk transactions can be processed seamlessly, improving both security and user experience.

The analysis also supports the development of adaptive systems that evolve with changing fraud patterns. By continuously updating detection strategies based on new data, financial institutions can stay ahead of fraudsters. Additionally, the insights emphasize the importance of multi-dimensional analysis, where decisions are based on a combination of features rather than a single variable.

Finally, the results highlight the need for real-time alert systems that can detect anomalies as they occur. This enables immediate response to suspicious activities, minimizing potential losses. Overall, the decision-making process should be driven by data insights, ensuring that fraud detection systems are both effective and adaptable.

9. CONCLUSION

9.1 SUMMARY OF THE PROJECT

This project presents a comprehensive analytical study of credit card fraud detection using data visualization and behavioral analytics. The analysis was conducted on a structured transaction dataset obtained from Kaggle, which includes customer details, transaction attributes, and geographical information. The workflow involved systematic steps such as data preprocessing, exploratory data analysis, feature engineering, and advanced analytical techniques including correlation analysis, feature selection, and dimensionality reduction.

Through the application of visualization techniques, the project successfully transformed raw transactional data into meaningful insights. Each stage of the workflow contributed to a deeper understanding of fraud patterns, highlighting how anomalies in transaction behavior can indicate fraudulent activity. The study emphasizes that fraud detection is not based on isolated variables but on identifying patterns across multiple dimensions such as time, amount, and category. Overall, the project demonstrates the effectiveness of data-driven analysis in uncovering hidden patterns in financial data.

9.2 OUTCOMES ACHIEVED

The project successfully identified important patterns associated with fraudulent transactions, including irregular spending behavior, time-based anomalies, and category-specific risks. Data preprocessing techniques ensured a clean and consistent dataset, while feature selection helped in focusing on the most relevant variables.

Additionally, the use of visualization techniques made it easier to interpret patterns and communicate insights effectively. The application of advanced analytical methods further enhanced the understanding of relationships between features. Overall, the project achieved its objective of extracting meaningful insights from transaction data to support fraud analysis.

9.3 LIMITATIONS

- The analysis is based on a static dataset and does not capture real-time transaction behavior.
- The dataset may not fully represent real-world complexities and evolving fraud strategies.
- The study focuses on data analysis and visualization, without implementing a real-time detection system.
- Limited availability of external factors (such as device data or user behavior logs) restricts deeper analysis.
- The imbalance in the dataset (very few fraud cases) makes it challenging to fully explore all fraud patterns.
- Geographical data is limited to coordinates and does not include detailed location context (e.g., country-level risk factors).

9.4 FUTURE SCOPE

Financial Applications

The insights from this project can be applied to develop real-time fraud monitoring systems that analyze transactions as they occur and identify suspicious activities more effectively.

Technological Advancements

Future work can include integrating adaptive systems and real-time data processing techniques to handle continuous transaction streams and evolving fraud patterns.

Research Opportunities

Further research can focus on sequence-based analysis and combining multiple data sources to improve the accuracy and depth of fraud detection.

10. REFERENCES

10.1 DATASET SOURCE

Kaggle: kartik2112/fraud-detection

This dataset contains structured credit card transaction records including transaction details, customer information, and geolocation data. It provides a comprehensive foundation for analyzing fraud patterns

10.2 TOOLS AND LIBRARIES USED

- **Pandas** – Used for data loading, cleaning, and manipulation.
- **NumPy** – Used for numerical operations and array computations.
- **Matplotlib** – Used for creating basic data visualizations.
- **Seaborn** – Used for advanced statistical and graphical visualizations.
- **Scikit-learn** – Used for feature selection (SelectKBest) and PCA.

10.3 RESEARCH PAPERS AND REFERENCES

1.Bolton, R. J., & Hand, D. J. (2002) – Statistical Fraud Detection: A Review

This paper explains statistical approaches for detecting fraud through anomaly and behavioral pattern analysis.

- Helped in understanding deviation-based analysis of transaction amount, time, and category.

2. Phua, C., Lee, V., Smith, K., & Gayler, R. (2010) – A Comprehensive Survey of Data Mining-based Fraud Detection Research

This survey discusses various data-driven techniques for identifying fraud using multiple features and patterns.

- Supported the multi-dimensional analysis approach used in EDA and feature relationships.

3. Chandola, V., Banerjee, A., & Kumar, V. (2009) – Anomaly Detection: A Survey

This paper provides a detailed overview of anomaly detection methods across different domains.

- Applied in identifying outliers and unusual transaction patterns using visualization techniques

4. Bahnsen, A. C., et al. (2016) – Feature Engineering Strategies for Credit Card Fraud Detection

This paper highlights the importance of selecting relevant features to improve fraud detection effectiveness.

- Guided the use of SelectKBest for identifying important features

5. Jurgovsky, J., et al. (2018) – Sequence Classification for Credit Card Fraud Detection

This paper focuses on analyzing transaction sequences and temporal behavior for fraud detection.

- Influenced the analysis of time-based patterns and transaction behavior trends.

APPENDIX

GITHUB LINK :

https://github.com/johiyasri/24ADI204_DSV_TEAM5.git

ONEDRIVE LINK :

https://kumaragurudtsteam-my.sharepoint.com/:f:/g/personal/divyadharshini_24ad_kct_ac_in/IgC92cd78AGISbJtfAVjGo-8AeuuuM8J-tW_j-Oj-vCvFsA?e=Zg6yc7

GOOGLE COLAB:

https://colab.research.google.com/drive/1Bn7kZ4iFlPoNn_QQ_yQhYN42k4VsOb3r?usp=sharing

CODE SNIPPETS

```
#WEEK 02 - KNOW YOUR DATA
print("Week : 02 ")

#LOADING THE DATASET
df = pd.read_csv("/kaggle/input/fraud-detection/fraudTrain.csv")

#PRINTING FIRST FIVE ROWS
print("\nFirst Five Rows:\n")
print(df.head())

#PRINTING LAST FIVE ROWS
print("\nLast Five Rows:\n")
print(df.tail())

#CHECKING THE DATA STRUCTURE
print("\nColumn Names:\n")
print(df.columns)
print("\nNumber of Rows and Columns:\n")
print(df.shape)

#CHECKING DATA TYPES
print("\nDataset Information:\n")
print(df.info())
print("\nData Types of Each Column:\n")
print(df.dtypes)

#STATISTICAL SUMMARY
print("\nStatistical Summary:\n")
print(df.describe())

#CHECKING FOR MISSING VALUES
print("\nMissing Values in Each Column:\n")
print(df.isnull().sum())
```

First Five Rows:

```
Unnamed: 0 trans_date_trans_time cc_num \
0 0 2019-01-01 00:00:18 2703186189652095
1 1 2019-01-01 00:00:44 630423337322
2 2 2019-01-01 00:00:51 38859492057661
3 3 2019-01-01 00:01:16 3534093764340240
4 4 2019-01-01 00:03:06 375534208663984

merchant category amt first \
0 fraud_Rippin, Kub and Mann misc_net 4.97 Jennifer
1 fraud_Heller, Gutmann and Zieme grocery_pos 107.23 Stephanie
2 fraud_Lind-Buckridge entertainment 220.11 Edward
3 fraud_Kutch, Hermiston and Farrell gas_transport 45.00 Jeremy
4 fraud_Keeling-Crist misc_pos 41.96 Tyler

last_gender street ... lat long \
0 Banks F 561 Perry Cove ... 36.0788 -81.1781
1 Gill F 43039 Riley Greens Suite 393 ... 48.8878 -118.2105
2 Sanchez M 594 White Dale Suite 530 ... 42.1808 -112.2620
3 White M 9443 Cynthia Court Apt. 038 ... 46.2306 -112.1138
4 Garcia M 408 Bradley Rest ... 38.4207 -79.4629

city_pop job dob \
0 3495 Psychologist, counselling 1988-03-09
1 149 Special educational needs teacher 1978-06-21
2 4154 Nature conservation officer 1962-01-19
3 1939 Patent attorney 1967-01-12
4 99 Dance movement psychotherapist 1986-03-28

trans_num unix_time merch_lat merch_long \
0 0b242abb623afc578575680df30655b9 1325376018 36.011293 -82.048315
1 1f76529f8574734946361c461b024d99 1325376044 49.159047 -118.186462
2 a1a22d70485983eac12b5b88dad1cf95 1325376051 43.150704 -112.154481
3 6b849c168bdad6f867558c3793159a81 1325376076 47.034331 -112.561071
4 a41d7549acf90789359a9aa5346dcb46 1325376186 38.674999 -78.632459

is_fraud
0 0
1 0
2 0
3 0
4 0
```

[5 rows x 23 columns]

Last Five Rows:

```
Unnamed: 0 trans_date_trans_time cc_num \
1296670 1296670 2020-06-21 12:12:08 30263540414123
1296671 1296671 2020-06-21 12:12:19 6011149206456997
1296672 1296672 2020-06-21 12:12:32 3514865930894695
1296673 1296673 2020-06-21 12:13:36 2720012583106919
1296674 1296674 2020-06-21 12:13:37 4292902571056973207

merchant category amt \
1296670 fraud_Reichel Inc entertainment 15.56
1296671 fraud_Abernathy and Sons food_dining 51.70
1296672 fraud_Stiedemann Ltd food_dining 105.93
1296673 fraud_Reinger, Weissnat and Strosin food_dining 74.90
1296674 fraud_Langosh, Wintheiser and Hyatt food_dining 4.30

first last_gender street ... \
1296670 Erik Patterson M 162 Jessica Row Apt. 072 ...
1296671 Jeffrey White M 8617 Holmes Terrace Suite 651 ...
1296672 Christopher Castaneda M 1632 Cohen Drive Suite 639 ...
1296673 Joseph Murray M 42933 Ryan Underpass ...
1296674 Jeffrey Smith M 135 Joseph Mountains ...

lat long city_pop job \
1296670 37.7175 -112.4777 258 Geoscientist
1296671 39.2667 -77.5101 100 Production assistant, television
1296672 32.9396 -105.8189 899 Naval architect
1296673 43.3526 -102.5411 1126 Volunteer coordinator
1296674 45.8433 -113.8748 218 Therapist, horticultural

dob trans_num unix_time merch_lat \
1296670 1961-11-24 440b587732da4dc1a6395aba5fb41669 1371816728 36.841266
1296671 1979-12-11 278000d2e0d2277d1de2f890067dcc0a 1371816739 38.906881
1296672 1967-08-30 483f52fe67fabef353d552c1e662974c 1371816752 33.619513
1296673 1980-08-18 d667cdcbadaaed3da3f4020e83591c83 1371816816 42.788940
1296674 1995-08-16 8f7c8e4ab7f25875d753b422917c98c9 1371816817 46.565983

merch_long is_fraud
1296670 -111.690765 0
1296671 -78.246528 0
1296672 -105.130529 0
1296673 -103.241160 0
1296674 -114.186110 0
```

[5 rows x 23 columns]

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129675 entries, 0 to 129674
Data columns (total 23 columns):
```

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	129675 non-null	int64
1	trans_date_trans_time	129675 non-null	object
2	cc_num	129675 non-null	int64
3	merchant	129675 non-null	object
4	category	129675 non-null	object
5	amt	129675 non-null	float64
6	first	129675 non-null	object
7	last	129675 non-null	object
8	gender	129675 non-null	object
9	street	129675 non-null	object
10	city	129675 non-null	object
11	state	129675 non-null	object
12	zip	129675 non-null	int64
13	lat	129675 non-null	float64
14	long	129675 non-null	float64
15	city_pop	129675 non-null	int64
16	job	129675 non-null	object
17	dob	129675 non-null	object
18	trans_num	129675 non-null	object
19	unix_time	129675 non-null	int64
20	merch_lat	129675 non-null	float64
21	merch_long	129675 non-null	float64
22	is_fraud	129675 non-null	int64

dtypes: float64(5), int64(6), object(12)

memory usage: 227.5+ MB

None

Data Types of Each Column:

Unnamed: 0	int64
trans_date_trans_time	object
cc_num	int64
merchant	object
category	object
amt	float64
first	object
last	object
gender	object
street	object
city	object
state	object
zip	int64
lat	float64
long	float64
city_pop	int64
job	object
dob	object
trans_num	object
unix_time	int64
merch_lat	float64
merch_long	float64
is_fraud	int64
dtype:	object

Statistical Summary:

	Unnamed: 0	cc_num	amt	zip	lat \
count	1.296675e+06	1.296675e+06	1.296675e+06	1.296675e+06	1.296675e+06
mean	6.483370e+05	4.171920e+17	7.035104e+01	4.880067e+04	3.853762e+01
std	3.743180e+05	1.308806e+18	1.603160e+02	2.689322e+04	5.075808e+00
min	0.000000e+00	6.041621e+10	1.000000e+00	1.257000e+03	2.002710e+01
25%	3.241685e+05	1.800429e+14	9.650000e+00	2.623700e+04	3.462050e+01
50%	6.483370e+05	3.521417e+15	4.752000e+01	4.817400e+04	3.935430e+01
75%	9.725055e+05	4.642255e+15	8.314000e+01	7.204200e+04	4.194040e+01
max	1.296674e+06	4.992346e+18	2.894890e+04	9.978300e+04	6.669330e+01

	long	city_pop	unix_time	merch_lat	merch_long \
count	1.296675e+06	1.296675e+06	1.296675e+06	1.296675e+06	1.296675e+06
mean	-9.022634e+01	8.882444e+04	1.349244e+09	3.853734e+01	-9.022646e+01
std	1.375908e+01	3.019564e+05	1.284128e+07	5.109788e+00	1.377109e+01
min	-1.656723e+02	2.300000e+01	1.325376e+09	1.902779e+01	-1.666712e+02
25%	-9.679800e+01	7.430000e+02	1.338751e+09	3.473357e+01	-9.689728e+01
50%	-8.747690e+01	2.456000e+03	1.349250e+09	3.936568e+01	-8.743839e+01
75%	-8.015800e+01	2.032800e+04	1.359385e+09	4.195716e+01	-8.023680e+01
max	-6.795030e+01	2.906700e+06	1.371817e+09	6.751027e+01	-6.695090e+01

	is_fraud
count	1.296675e+06
mean	5.788652e-03
std	7.586269e-02
min	0.000000e+00
25%	0.000000e+00
50%	0.000000e+00
75%	0.000000e+00
max	1.000000e+00

Missing Values in Each Column:

Unnamed: 0	0
trans_date_trans_time	0
cc_num	0
merchant	0
category	0
amt	0
first	0
last	0
gender	0
street	0
city	0
state	0
zip	0
lat	0
long	0
city_pop	0
job	0
dob	0
trans_num	0
unix_time	0
merch_lat	0
merch_long	0
is_fraud	0
dtype:	int64

INTERPRETATION :

- The dataset contains 1,296,675 transaction records with 23 attributes, indicating a large dataset suitable for fraud detection analysis.
- The dataset includes a mix of numerical attributes (such as amt, city_pop, lat, long, unix_time) and categorical attributes (such as merchant, category, gender, city, and job). This combination provides detailed information about both transaction characteristics and customer demographics.
- The statistical summary shows that the average transaction amount is approximately 70.35, while the maximum transaction amount is significantly higher, indicating the presence of high-value transactions that may require further analysis.
- The target variable is _fraud contains values 0 and 1, where 0 represents legitimate transactions and 1 represents fraudulent transactions. The mean value of this column is very low (around 0.0057), which indicates that fraudulent transactions form only a small proportion of the dataset, highlighting the class imbalance problem.
- The dataset inspection also shows that no missing values are present in any of the columns, indicating that the dataset is complete and suitable for further preprocessing and analysis .

```
print("Week - 03")
#THE CLEANING SPRINT

#INTRODUCING MISSING VALUES IN THE DATASET AS THE DATASET HAS NO MISSING VALUES
np.random.seed(42)
df.loc[np.random.choice(df.index, 2000, replace=False), 'amt'] = np.nan
df.loc[np.random.choice(df.index, 1500, replace=False), 'city_pop'] = np.nan
df.loc[np.random.choice(df.index, 1000, replace=False), 'merchant'] = np.nan
df.loc[np.random.choice(df.index, 1000, replace=False), 'job'] = np.nan

df = df.drop(columns=['Unnamed: 0'], errors='ignore')
print("Dataset Shape:", df.shape)
missing_counts = df.isnull().sum()

print("\nMissing Values Before Imputation:\n")
print(missing_counts)

#MISSING VALUES BEFORE IMPUTATION (BAR CHART)
if missing_counts.sum() > 0:
    plt.figure(figsize=(12,5))
    missing_counts.plot(kind='bar')
    plt.title("Missing Values Count (Before Imputation)")
    plt.ylabel("Count")
    plt.xticks(rotation=45)
    plt.show()

if missing_counts.sum() > 0:
    plt.figure(figsize=(12,5))
    missing_counts[missing_counts > 0].plot(kind='bar')
    plt.title("Missing Values Count (Before Imputation)")
    plt.ylabel("Count")
    plt.xticks(rotation=45)
    plt.show()
```

```

num_cols = df.select_dtypes(include=np.number).columns
cat_cols = df.select_dtypes(exclude=np.number).columns

#HANDLING MISSING VALUES FOR NUMERICAL COLUMNS USING MEDIAN IMPUTATION
for col in num_cols:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())

#HANDLING MISSING VALUES FOR CATEGORICAL COLUMNS USING MODE IMPUTATION
for col in cat_cols:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].value_counts().idxmax())

print("\nMissing Values After Imputation:", df.isnull().sum().sum())
missing_counts1 = df.isnull().sum().sum()
print(df.isnull().sum())

#OUTLIER DETECTION USING IQR METHOD
if 'amt' in df.columns:
    Q1 = df['amt'].quantile(0.25)
    Q3 = df['amt'].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df['amt'] < lower) | (df['amt'] > upper)]
    print("\nOutlier Detection using IQR")
    print("Lower Bound:", lower)
    print("Upper Bound:", upper)
    print("Number of Outliers:", outliers.shape[0])
    plt.figure(figsize=(8,4))
    sns.boxplot(x=df['amt'])
    plt.title("Outliers in Transaction Amount (Before Scaling)")
    plt.show()

print("\n\n")

```

```

#OUTLIER DETECTION USING Z-SCORE METHOD
if 'amt' in df.columns:
    z_scores = np.abs(zscore(df['amt']))
    # Threshold = 3
    outliers_zscore = df[z_scores > 3]
    print("\nOutlier Detection using Z-Score Method")
    print("Z-score Threshold: 3")
    print("Number of Outliers (Z-score):", outliers_zscore.shape[0])

#VISUALIZATION FOR Z-SCORE
plt.figure(figsize=(8,4))
sns.histplot(z_scores, bins=50)
plt.title("Z-Score Distribution for Transaction Amount")
plt.xlabel("Z-Score")
plt.ylabel("Frequency")
plt.show()

```

```

#SCALING USING ROBUST SCALAR
if 'amt' in df.columns:
    scaler = RobustScaler()
    df['amt_scaled'] = scaler.fit_transform(df[['amt']])

# AFTER SCALING
plt.figure(figsize=(8,4))
sns.boxplot(x=df['amt_scaled'])
plt.title("After Robust Scaling")
plt.show()
print("\n\n")

# DISTRIBUTION COMPARISON
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
sns.histplot(df['amt'], bins=30)
plt.title("Before Scaling")
plt.subplot(1,2,2)
sns.histplot(df['amt_scaled'], bins=30)
plt.title("After Robust Scaling")
plt.tight_layout()
plt.show()

for col in cat_cols:
    df[col] = df[col].astype('category')
import gc
gc.collect()

#HISTOGRAM
plt.figure(figsize=(8,4))
sns.histplot(df['amt_scaled'], bins=700)
plt.xlim(-5, 5)
plt.title("Scaled Amount (Zoomed)")
plt.show()

```

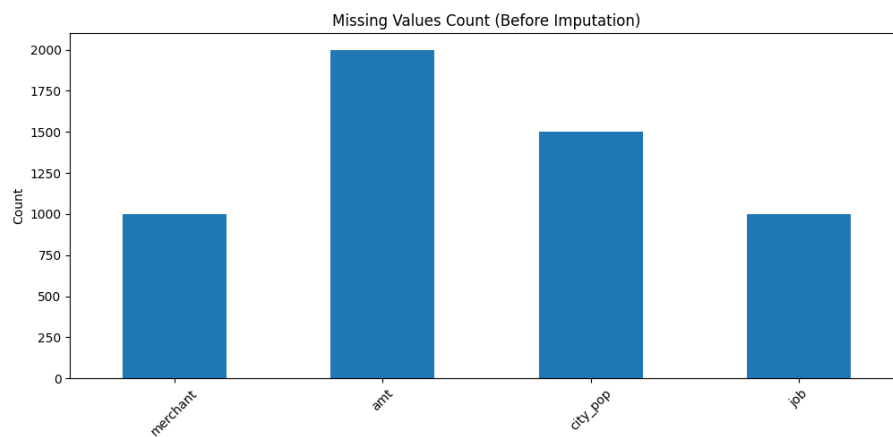
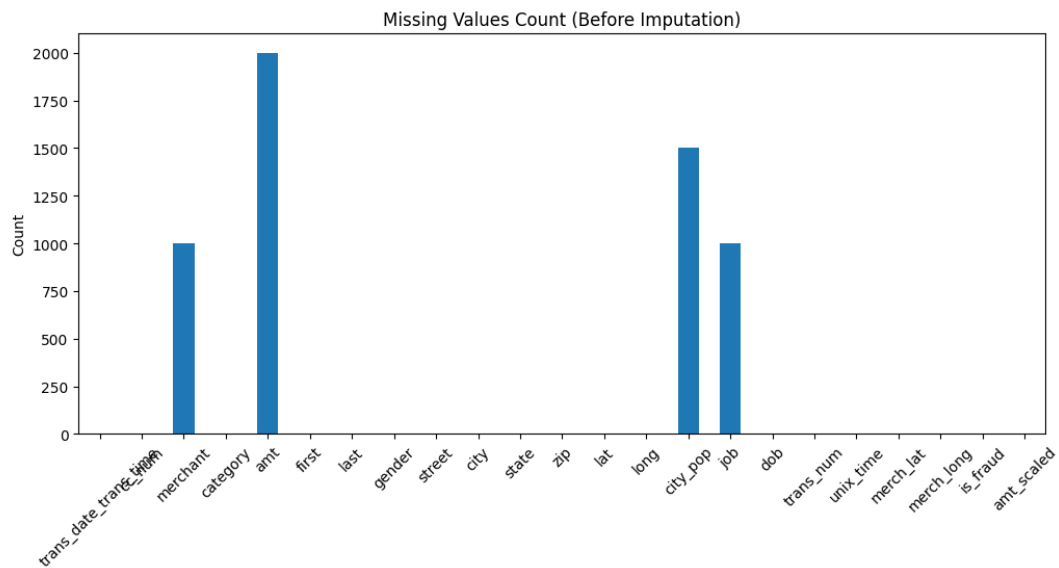
OUTPUT :

Week - 03

Dataset Shape: (1296675, 23)

Missing Values Before Imputation:

trans_date_trans_time	0
cc_num	0
merchant	1000
category	0
amt	2000
first	0
last	0
gender	0
street	0
city	0
state	0
zip	0
lat	0
long	0
city_pop	1500
job	1000
dob	0
trans_num	0
unix_time	0
merch_lat	0
merch_long	0
is_fraud	0
amt_scaled	0
dtype: int64	



Missing Values After Imputation: 0

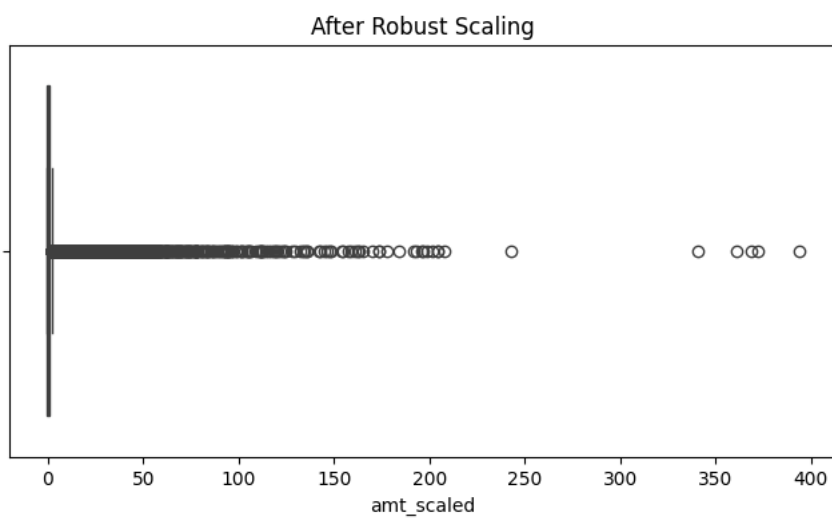
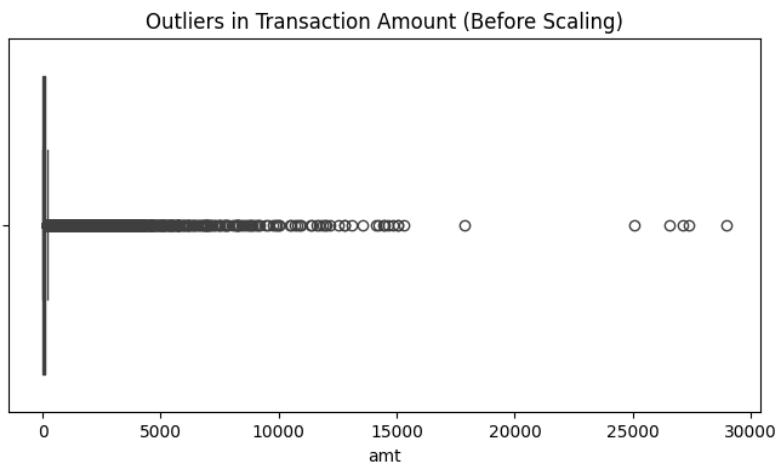
```
trans_date_trans_time    0
cc_num                   0
merchant                  0
category                  0
amt                       0
first                     0
last                      0
gender                    0
street                    0
city                      0
state                     0
zip                       0
lat                       0
long                      0
city_pop                  0
job                        0
dob                       0
trans_num                 0
unix_time                 0
merch_lat                 0
merch_long                0
is_fraud                  0
amt_scaled                0
dtype: int64
```

Outlier Detection using IQR

Lower Bound: -100.415

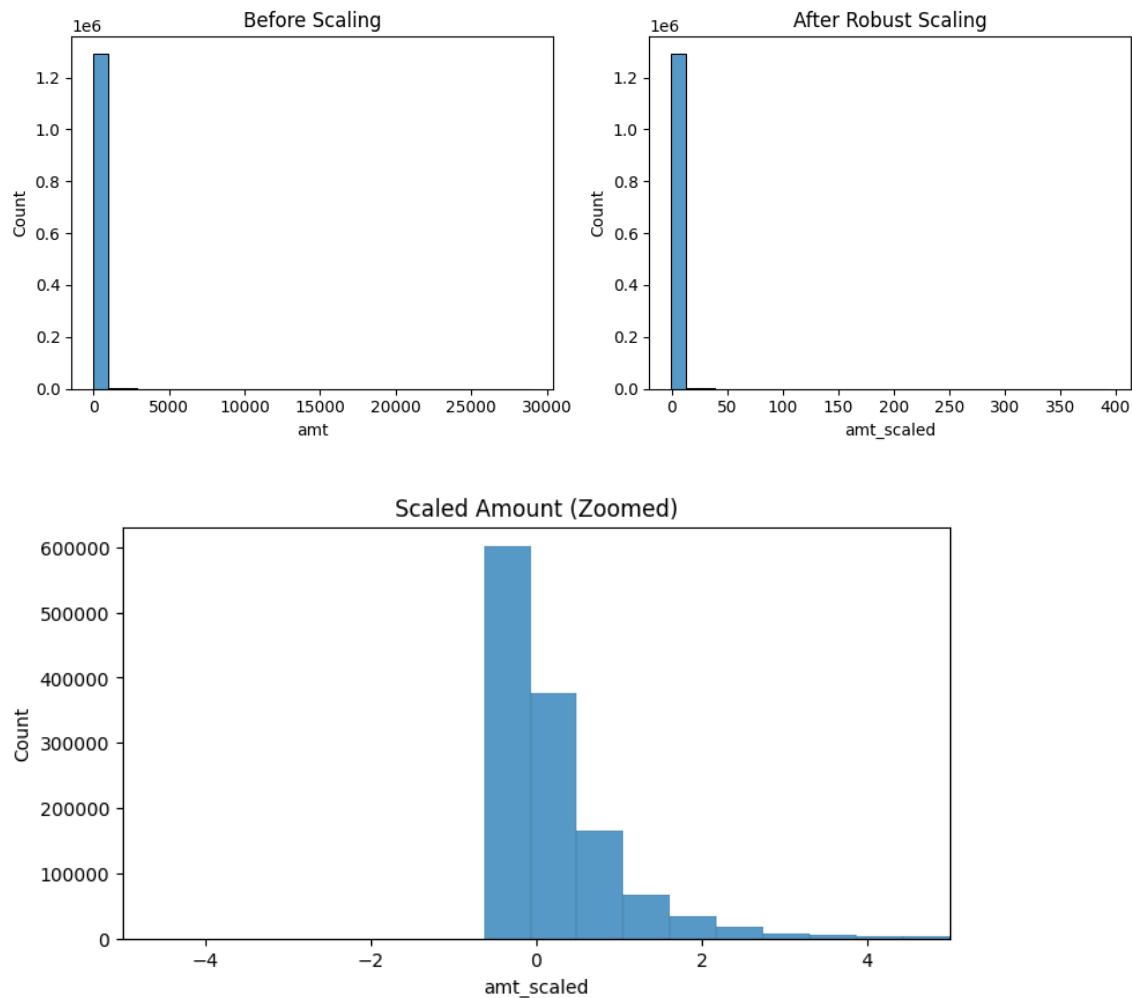
Upper Bound: 193.145

Number of Outliers: 67386



Outlier Detection using Z-Score Method
Z-score Threshold: 3
Number of Outliers (Z-score): 12731





INTERPRETATION :

- Missing values were intentionally introduced into selected columns (amt, city_pop, merchant, and job) to demonstrate data cleaning techniques. A total of 5,500 missing values were identified in the dataset.
- Median imputation was applied to numerical features such as amt and city_pop, while mode imputation was applied to categorical features such as merchant and job. After applying these imputation strategies, all missing values were successfully removed from the dataset.
- Outlier detection using the Interquartile Range (IQR) method revealed that the transaction amount feature contains 67,386 outliers. The calculated lower and upper bounds were -100.415 and 193.145 , respectively. These results indicate that many transactions have values significantly higher than the normal spending range.

- Outlier detection using the Z-Score method was also performed to measure how far each transaction amount deviates from the mean in terms of standard deviations. Transactions with a Z-Score greater than ± 3 were classified as outliers. This method provided an additional statistical validation for identifying extreme values.
- The presence of these extreme values suggests that certain transactions are unusually large compared to typical spending behavior, which may indicate rare high-value purchases or potentially fraudulent activities.
- Visualization techniques such as boxplots and distribution plots helped in clearly identifying the spread of transaction amounts and the presence of extreme values.
- To reduce the impact of these outliers on further analysis, Robust Scaling was applied to the amt column. This technique uses the median and interquartile range, making it effective for datasets containing extreme values.

EDA DEEP DIVE

```
print("Week - 04")
# EXPLORATORY DATA ANALYSIS (EDA) - DEEP DIVE

sns.set_style("whitegrid")
sns.set_context("talk")

# Feature Engineering
# Extract hour from transaction timestamp
df['trans_date_trans_time'] = pd.to_datetime(df['trans_date_trans_time'])
df['hour'] = df['trans_date_trans_time'].dt.hour

# Sample data for faster visualization
df_sample = df.sample(50000, random_state=42)

print("\n===== UNIVARIATE ANALYSIS =====\n")

# Distribution of Transaction Amount (Log Scale)
plt.figure(figsize=(10,6))
sns.histplot(np.log1p(df_sample['amt']),
             bins=20,
             kde=True,
             color="#1f77b4")
plt.title("Distribution of Transaction Amount (Log Scale)", fontsize=16)
plt.xlabel("Log(Transaction Amount)", fontsize=13)
plt.ylabel("Frequency", fontsize=13)
plt.grid(True)
plt.tight_layout()
plt.show()
print("\n")

# Boxplot of Transaction Amount
plt.figure(figsize=(8,5))
sns.boxplot(y=np.log1p(df_sample['amt']),
            color="#ff7f0e")
plt.title("Boxplot of Transaction Amount (Log Scale)", fontsize=16)
plt.ylabel("Log(Transaction Amount)", fontsize=13)
plt.grid(True)
plt.tight_layout()
plt.show()
```

```

# Transaction Frequency by Hour
plt.figure(figsize=(10,6))
sns.countplot(x='hour',
              data=df_sample,
              palette="viridis")
plt.title("Transaction Frequency by Hour of Day", fontsize=16)
plt.xlabel("Hour of Day", fontsize=13)
plt.ylabel("Number of Transactions", fontsize=13)
plt.grid(True)
plt.tight_layout()
plt.show()
print("\n")

# BIVARIATE ANALYSIS
print("\n===== BIVARIATE ANALYSIS =====\n")

# Fraud vs Transaction Amount
plt.figure(figsize=(10,6))
sns.boxplot(x='is_fraud',
            y=np.log1p(df_sample['amt']),
            data=df_sample,
            palette=["#2ca02c", "#d62728"])
plt.title("Fraud vs Transaction Amount (Log Scale)", fontsize=16)
plt.xlabel("Fraud Label (0 = Legitimate, 1 = Fraud)", fontsize=13)
plt.ylabel("Log(Transaction Amount)", fontsize=13)
plt.grid(True)
plt.tight_layout()
plt.show()
print("\n")

#Transaction Amount vs Hour
plt.figure(figsize=(10,6))
sns.scatterplot(x='hour',
                y=np.log1p(df_sample['amt']),
                hue='is_fraud',
                data=df_sample,
                alpha=0.5,
                palette={0:"#1f77b4",1:"#d62728"})
plt.title("Transaction Amount vs Hour of Day", fontsize=16)

```



```

plt.xlabel("Hour of Day", fontsize=13)
plt.ylabel("Log(Transaction Amount)", fontsize=13)
plt.legend(title="Fraud")
plt.grid(True)
plt.tight_layout()
plt.show()
print("\n")

# Fraud vs Non-Fraud Transactions
plt.figure(figsize=(8,5))
sns.countplot(x='is_fraud',
              data=df_sample,
              palette=["green", "red"])
plt.title("Fraud vs Non-Fraud Transaction Distribution", fontsize=16)
plt.xlabel("Fraud Label (0 = Legitimate, 1 = Fraud)", fontsize=13)
plt.ylabel("Transaction Count", fontsize=13)
plt.grid(True)
plt.tight_layout()
plt.show()
print("\n")

# Fraud Rate by Hour
fraud_hour = df_sample.groupby('hour')['is_fraud'].mean().reset_index()
plt.figure(figsize=(10,6))
sns.lineplot(x='hour',
             y='is_fraud',
             data=fraud_hour,
             marker='o',
             color="red")
plt.title("Fraud Rate by Hour of Day", fontsize=16)
plt.xlabel("Hour of Day", fontsize=13)
plt.ylabel("Fraud Rate", fontsize=13)
plt.grid(True)
plt.tight_layout()
plt.show()
print("\n")

```

```

# MULTIVARIATE ANALYSIS
print("\n===== MULTIVARIATE ANALYSIS =====\n")

# Correlation Heatmap
numeric_cols = df.select_dtypes(include=np.number).columns
plt.figure(figsize=(12,8))
corr_matrix = df[numeric_cols].corr()
sns.heatmap(corr_matrix,
            cmap="coolwarm",
            linewidths=0.5,
            center=0,
            cbar_kws={"shrink":0.8})
plt.title("Correlation Heatmap of Numerical Features", fontsize=16)
plt.tight_layout()
plt.show()
print("\n")

# Category Distribution
plt.figure(figsize=(12,6))
sns.countplot(y='category',
              data=df_sample,
              order=df_sample['category'].value_counts().index)
plt.title("Transaction Distribution by Category", fontsize=16)
plt.xlabel("Transaction Count", fontsize=13)
plt.ylabel("Category", fontsize=13)
plt.tight_layout()
plt.show()
print("\n")

# Gender vs Fraud
plt.figure(figsize=(8,5))
sns.countplot(x='gender',
              hue='is_fraud',
              data=df_sample)
plt.title("Fraud Distribution by Gender", fontsize=16)
plt.xlabel("Gender", fontsize=13)
plt.ylabel("Transaction Count", fontsize=13)
plt.xticks(ticks=[0,1], labels=['Female','Male'], fontsize=12)
plt.tight_layout()
plt.show()

```

```

# Violin Plot - Fraud vs Amount
plt.figure(figsize=(10,6))
sns.violinplot(x='is_fraud',
               y=np.log1p(df_sample['amt']),
               data=df_sample,
               palette="Set2",
               inner="quartile")

plt.title("Transaction Amount Distribution by Fraud Status", fontsize=16)
plt.xlabel("Fraud Label", fontsize=13)
plt.ylabel("Log(Transaction Amount)", fontsize=13)
plt.tight_layout()
plt.show()
print("\n")

# Violin Plot - Amount by Hour
plt.figure(figsize=(12,6))
sns.violinplot(x='hour',
               y=np.log1p(df_sample['amt']),
               data=df_sample,
               palette="Spectral")

plt.title("Transaction Amount Distribution Across Hours", fontsize=16)
plt.xlabel("Hour of Day", fontsize=13)
plt.ylabel("Log(Transaction Amount)", fontsize=13)
plt.tight_layout()
plt.show()
print("\n")

# KDE Density Plot
plt.figure(figsize=(10,6))
sns.kdeplot(np.log1p(df_sample[df_sample['is_fraud']==0]['amt']),
            label="Legitimate",
            fill=True,
            color="green")
sns.kdeplot(np.log1p(df_sample[df_sample['is_fraud']==1]['amt']),
            label="Fraud",
            fill=True,
            color="red")

plt.title("Density Distribution of Transaction Amount", fontsize=16)
plt.xlabel("Log(Transaction Amount)", fontsize=13)
plt.ylabel("Density", fontsize=13)
plt.legend()

```

```

plt.tight_layout()
plt.show()
print("\n")

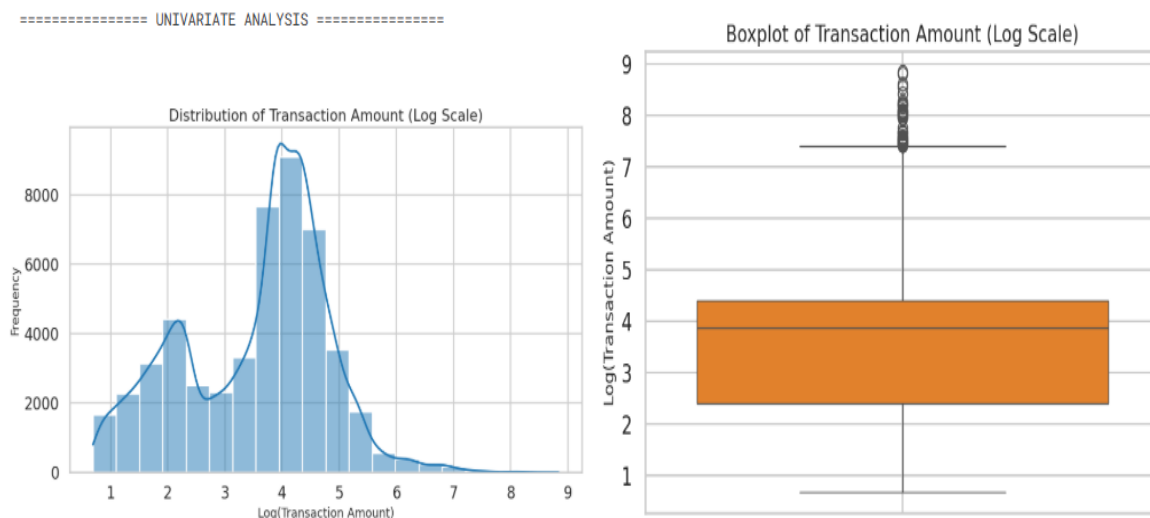
# Candlestick Visualization (Only 2 Months)
df_candle = df_sample.copy()
df_candle['trans_date_trans_time'] = pd.to_datetime(df_candle['trans_date_trans_time'])
df_candle = df_candle[
    (df_candle['trans_date_trans_time'] >= '2019-01-01') &
    (df_candle['trans_date_trans_time'] <= '2019-02-28')
]
df_candle.set_index('trans_date_trans_time', inplace=True)
ohlcv = df_candle['amt'].resample('1H').agg(['first', 'max', 'min', 'last']).dropna()
ohlcv.columns = ['open', 'high', 'low', 'close']
fig = go.Figure(data=[go.Candlestick(
    x=ohlcv.index,
    open=ohlcv['open'],
    high=ohlcv['high'],
    low=ohlcv['low'],
    close=ohlcv['close'],
    increasing_line_color='green',
    decreasing_line_color='red'
)])

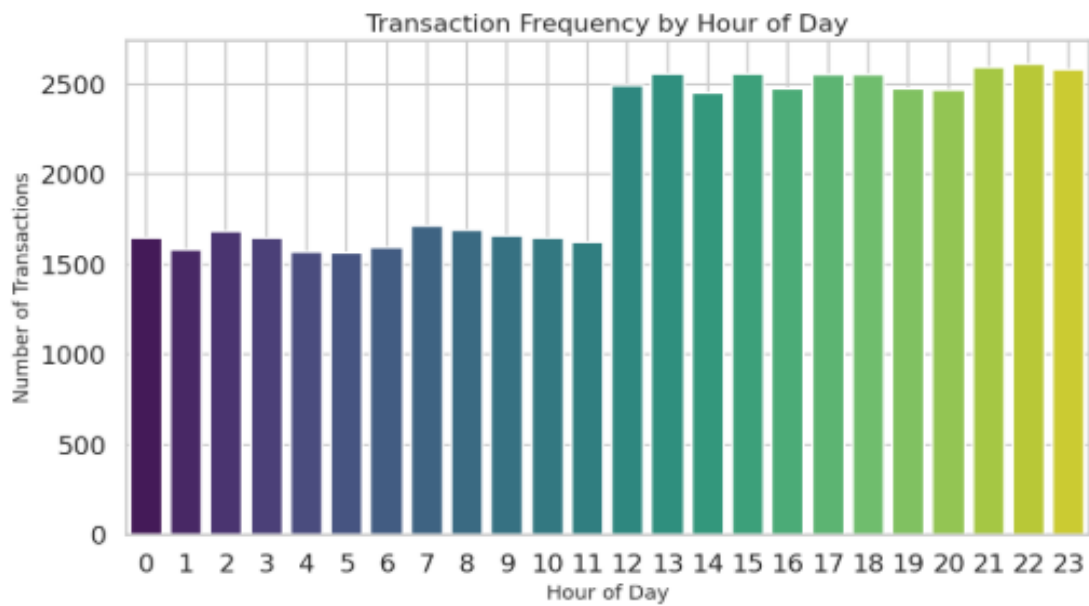
fig.update_layout(
    title="Transaction Amount Movement Over Time (Jan-Feb 2019)",
    xaxis_title="Transaction Time",
    yaxis_title="Transaction Amount",
    template="plotly_dark"
)

fig.show()

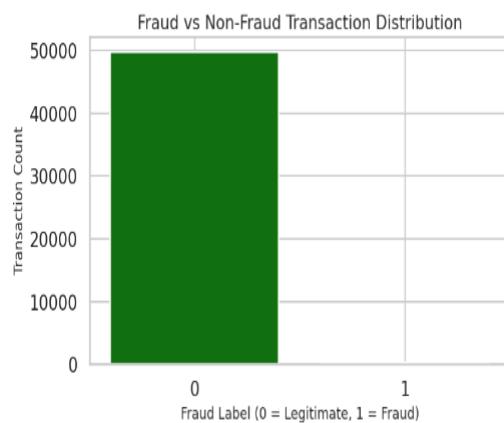
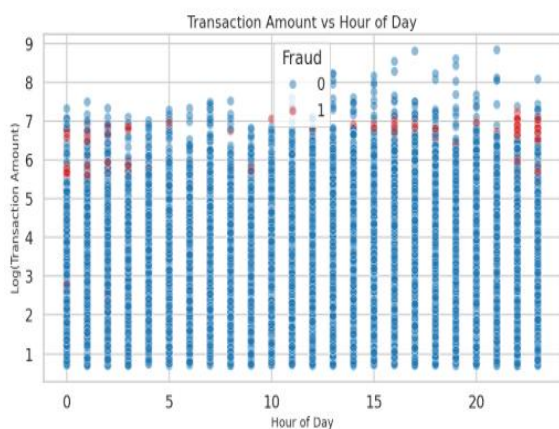
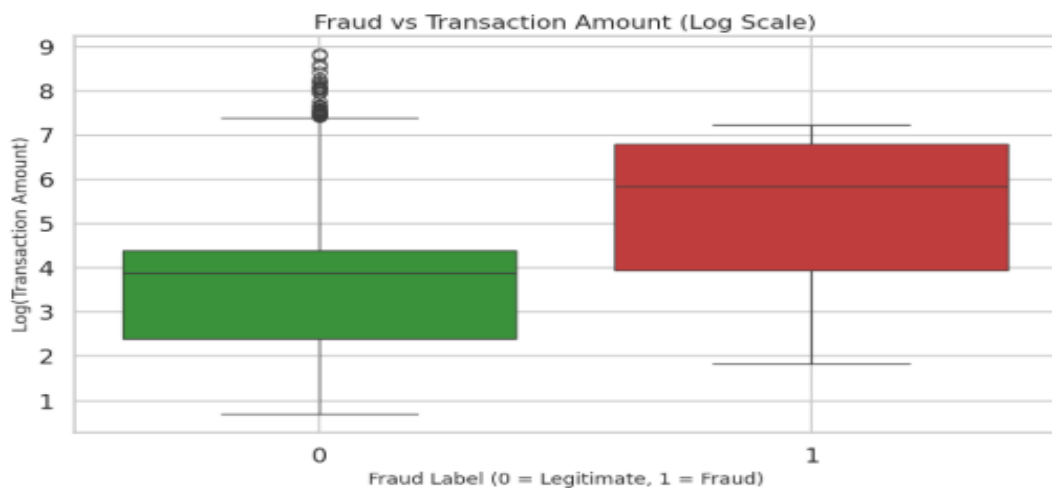
```

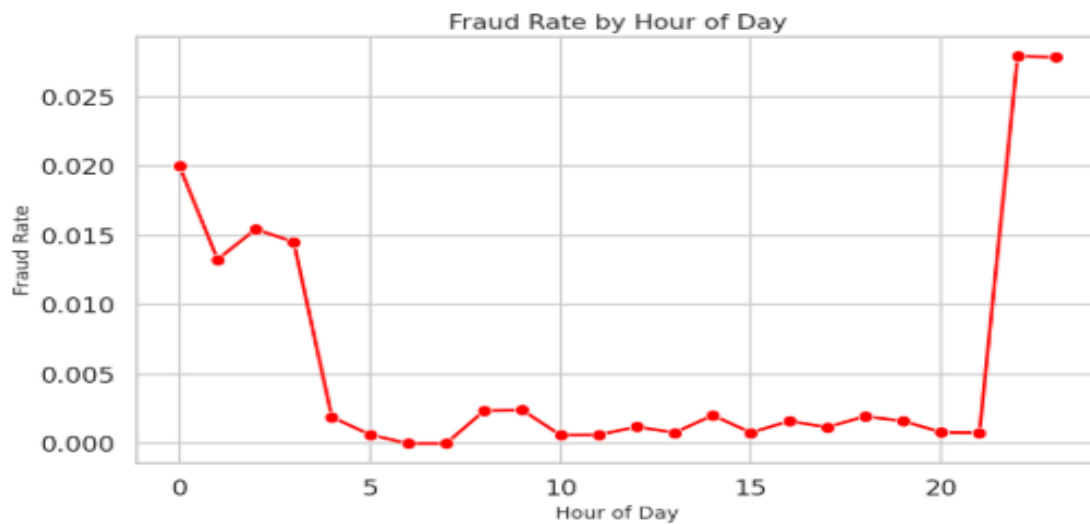
OUTPUT:



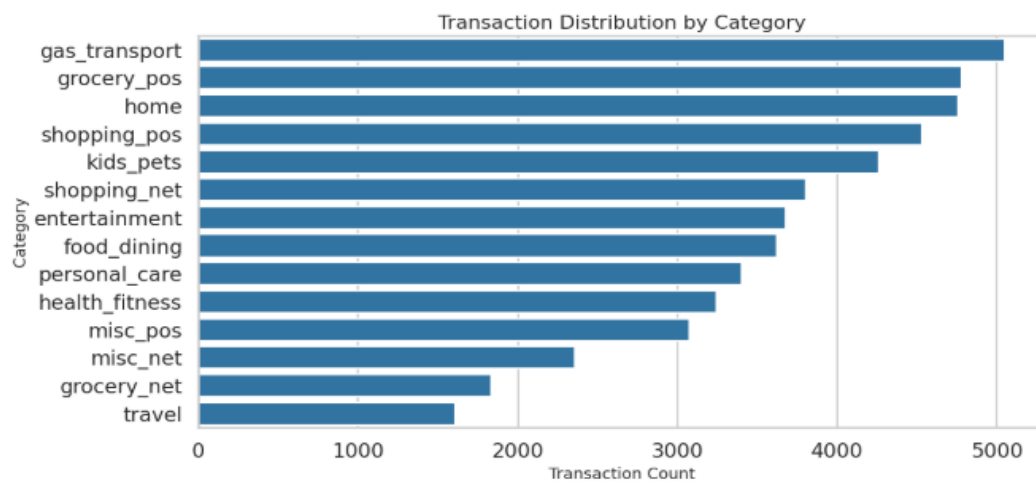
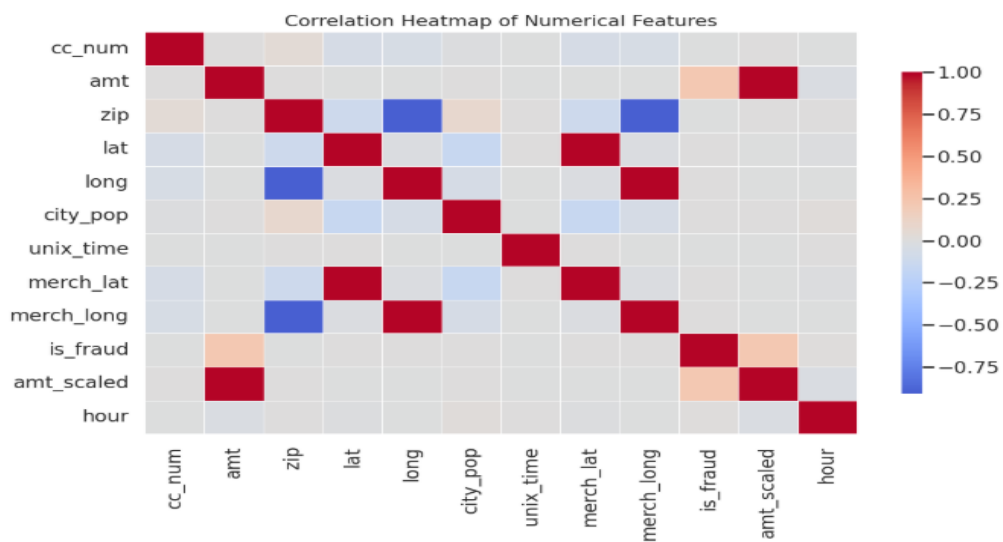


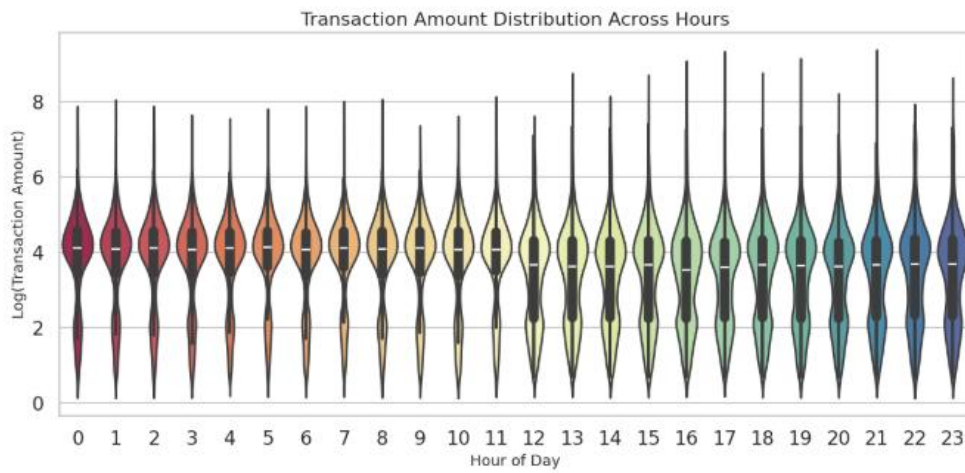
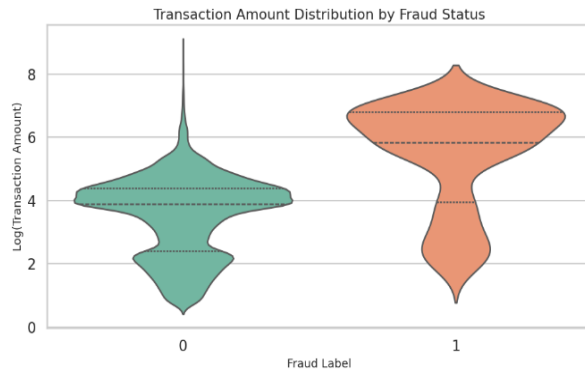
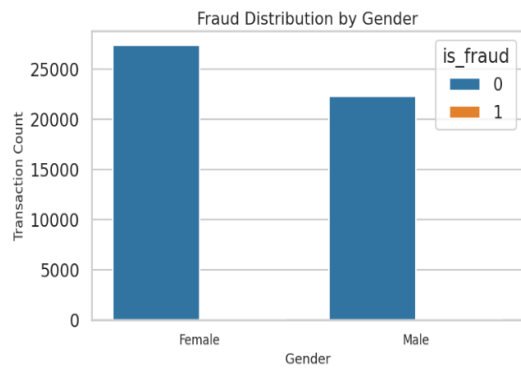
===== BIVARIATE ANALYSIS =====

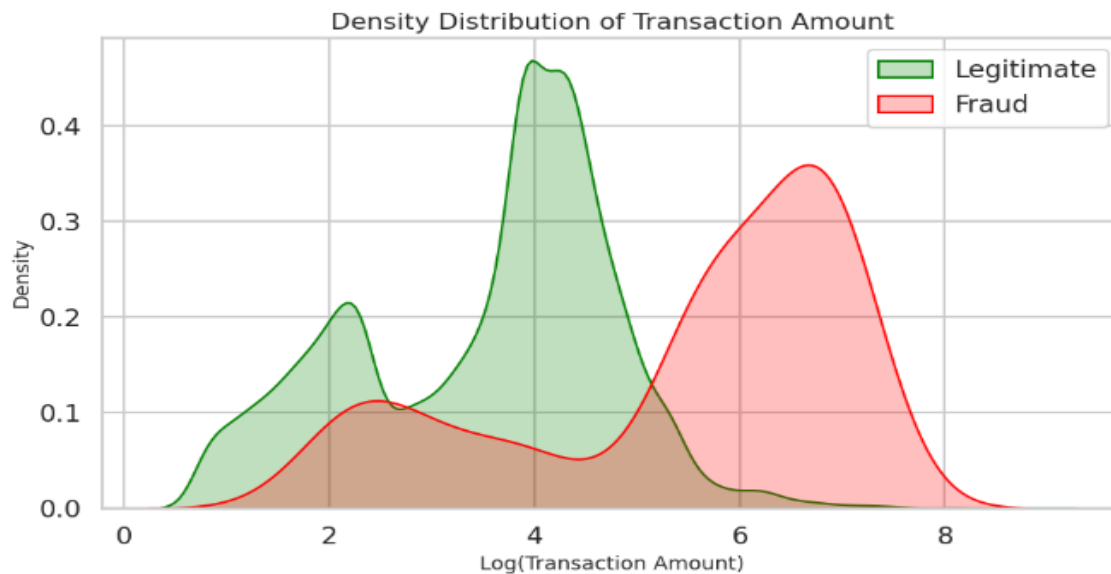




===== MULTIVARIATE ANALYSIS =====







INTERPRETATION:

- Exploratory Data Analysis (EDA) was conducted to understand the distribution and patterns in the transaction dataset using univariate, bivariate, and multivariate techniques.
- The transaction amount (log scale) shows a right-skewed distribution, indicating most transactions are low-value with a few high-value outliers.
- Boxplot analysis confirms the presence of extreme outliers, representing unusually large transactions.
- Transaction frequency varies by hour, showing peak activity at certain times and low activity during late hours.
- Fraud analysis reveals that fraudulent transactions generally have higher amounts and greater variability compared to legitimate ones.
- The dataset is highly imbalanced, with very few fraud cases compared to non-fraud cases.
- Fraud rate varies across hours, indicating that time is an important factor in detecting fraud.
- Correlation analysis shows low relationships between features, reducing multicollinearity issues.

- Visualization techniques like violin plots and KDE plots indicate that fraud transactions are more spread out and tend to have higher values.
- The candlestick plot shows transaction amount fluctuations over time, helping identify trends and unusual spikes.

FEATURE ENGINEERING

```
print("Week - 06")
#Feature Engineering

sns.set_style("whitegrid")
# Identify Numerical Columns
numeric_cols = df.select_dtypes(include=np.number).columns

# Remove target column from transformation
if 'is_fraud' in numeric_cols:
    numeric_cols = numeric_cols.drop('is_fraud')
print("\nNumerical Columns used for Feature Engineering:")
print(numeric_cols)

# Detect Skewness in Numerical Features
print("\nChecking Skewness of Numerical Features")
skew_values = df[numeric_cols].apply(lambda x: skew(x.dropna()))
print("\nSkewness Before Transformation:")
print(skew_values.sort_values(ascending=False))

# Fix Skewness using Log Transformation
print("\nApplying Log Transformation for Highly Skewed Features")
for col in numeric_cols:
    if abs(skew(df[col].dropna())) > 1:
        if (df[col] <= 0).any():
            df[col] = df[col] - df[col].min() + 1
        df[col] = np.log1p(df[col])

# Encode Categorical Variables
print("\nEncoding Categorical Variables")
cat_cols = df.select_dtypes(include=['object', 'category']).columns
label_encoder = LabelEncoder()
for col in cat_cols:
    df[col] = label_encoder.fit_transform(df[col])

# Apply Standard Scaling
print("\nApplying Standard Scaling")
standard_scaler = StandardScaler()
df_standard_scaled = df.copy()
df_standard_scaled[numeric_cols] = standard_scaler.fit_transform(df[numeric_cols])
print("\nSample Standard Scaled Data:")
```

```

print(df_standard_scaled[numeric_cols].head())

# Apply Min-Max Normalization
print("\nApplying Min-Max Normalization")
minmax_scaler = MinMaxScaler()
df_normalized = df.copy()
df_normalized[numeric_cols] = minmax_scaler.fit_transform(df[numeric_cols])
print("\nSample Normalized Data:")
print(df_normalized[numeric_cols].head())

# Visualization : Distribution After Skewness Correction
if 'amt' in df.columns:
    plt.figure(figsize=(8,5))
    sns.histplot(df['amt'], bins=40, kde=True, color="teal")
    plt.title("Transaction Amount Distribution After Skewness Correction", fontsize=14)
    plt.xlabel("Transaction Amount")
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.show()

# Visualization : Example Feature After Standard Scaling
plt.figure(figsize=(8,5))
sns.histplot(df_standard_scaled[numeric_cols[0]], bins=40, kde=True, color="blue")
plt.title("Example Feature Distribution After Standard Scaling")
plt.xlabel("Scaled Value")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# Visualization : Example Feature After Min-Max Normalization
plt.figure(figsize=(8,5))
sns.histplot(df_normalized[numeric_cols[0]], bins=40, kde=True, color="purple")
plt.title("Example Feature Distribution After Min-Max Normalization")
plt.xlabel("Normalized Value (0 to 1)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

```

```

# Save Feature Engineered Dataset
df_normalized.to_csv("fraud_feature_engineered_dataset.csv", index=False)

print("\nFeature Engineering Completed Successfully")
print("Final dataset saved as : fraud_feature_engineered_dataset.csv")

```

OUTPUT:

Numerical Columns used for Feature Engineering:

```
Index(['cc_num', 'amt', 'zip', 'lat', 'long', 'city_pop', 'unix_time',  
      'merch_lat', 'merch_long', 'amt_scaled', 'hour'],  
      dtype='object')
```

Checking Skewness of Numerical Features

Skewness Before Transformation:

```
amt          42.334251  
amt_scaled   42.334251  
city_pop     5.597503  
cc_num       2.851876  
zip          0.079681  
unix_time    0.003378  
merch_lat    -0.181915  
lat          -0.186027  
hour         -0.282825  
merch_long   -1.146959  
long         -1.150106  
dtype: float64
```

Applying Log Transformation for Highly Skewed Features

Encoding Categorical Variables

Applying Standard Scaling

Sample Standard Scaled Data:

	cc_num	amt	zip	lat	long	city_pop	unix_time	\
0	0.257715	-1.356008	-0.749136	-0.484420	0.542295	-0.082289	-1.858664	
1	-1.882392	0.892683	1.872567	2.039120	-1.741927	-1.367366	-1.858662	
2	-0.827809	1.447113	1.281042	0.717754	-1.277769	-0.011809	-1.858662	
3	0.326299	0.228658	0.402753	1.515617	-1.266852	-0.322646	-1.858660	
4	-0.247360	0.175597	-0.906090	-0.023035	0.622551	-1.532846	-1.858651	

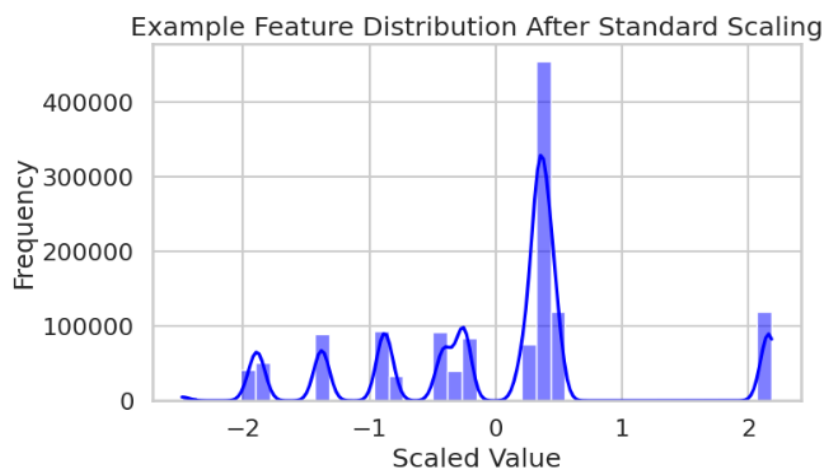
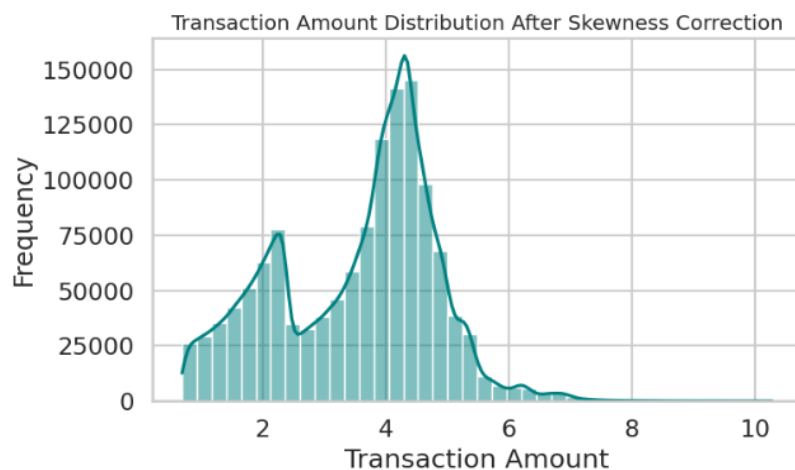
	merch_lat	merch_long	amt_scaled	hour
0	-0.494354	0.511873	-0.925790	-1.878145
1	2.078699	-1.784186	0.723404	-1.878145
2	0.902849	-1.304191	1.898349	-1.878145
3	1.662886	-1.334897	-0.175737	-1.878145
4	0.026941	0.676354	-0.226897	-1.878145

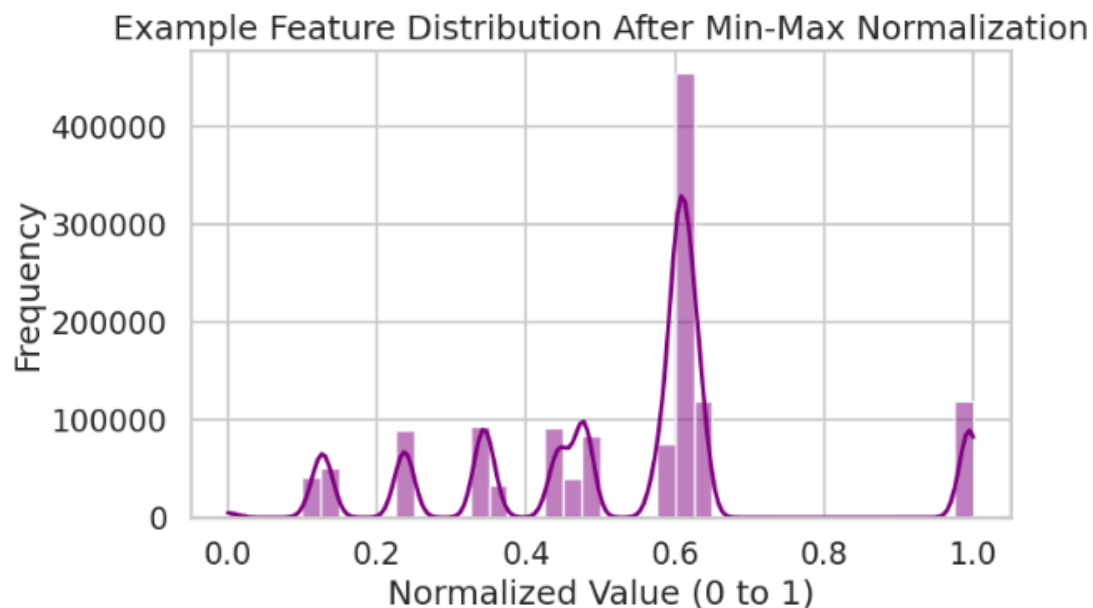
Applying Min-Max Normalization

Sample Normalized Data:

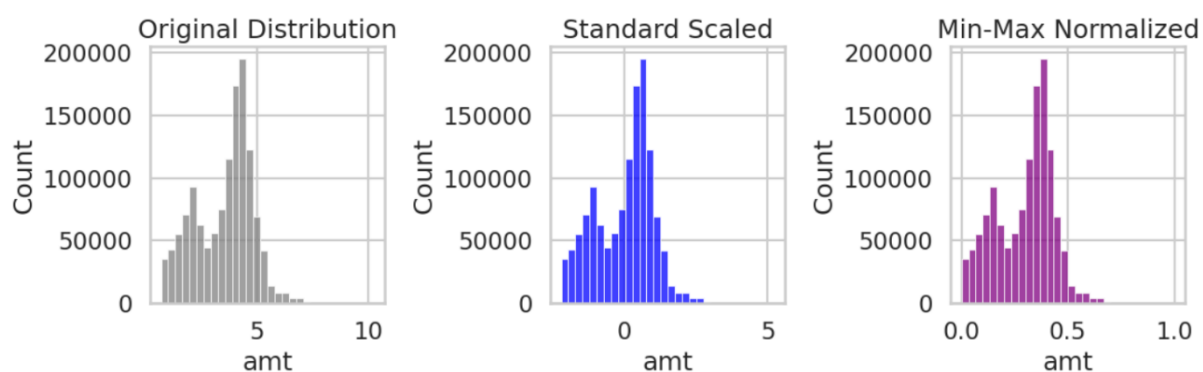
	cc_num	amt	zip	lat	long	city_pop	unix_time \
0	0.587424	0.114152	0.278069	0.343968	0.963597	0.425591	0.000000e+00
1	0.128642	0.416601	0.993677	0.618450	0.820634	0.156571	5.598526e-07
2	0.354716	0.491172	0.832217	0.474727	0.849684	0.440346	7.105821e-07
3	0.602126	0.327290	0.592483	0.561509	0.850367	0.375274	1.248902e-06
4	0.479149	0.320153	0.235227	0.394153	0.968620	0.121929	3.617509e-06

	merch_lat	merch_long	amt_scaled	hour
0	0.350302	0.959109	0.005046	0.0
1	0.621488	0.821700	0.102941	0.0
2	0.497559	0.850426	0.172685	0.0
3	0.577663	0.848588	0.049568	0.0
4	0.405244	0.968953	0.046532	0.0





Feature Engineering Completed Successfully
 Final dataset saved as : fraud_feature_engineered_dataset.csv



INTERPRETATION:

- Feature engineering was performed to improve data quality by transforming numerical and categorical variables for better model performance.
- Skewness analysis revealed that several numerical features were highly skewed, indicating an uneven distribution of values.
- Log transformation was applied to highly skewed features, which helped in reducing skewness and making the data more normally distributed.
- Categorical variables were encoded using label encoding, converting non-numeric data into machine-readable numerical form.

- Standard scaling was applied to normalize features with mean 0 and standard deviation 1, ensuring that all features contribute equally to the model.
- Min-Max normalization scaled the data into a fixed range (0 to 1), preserving the relative differences between values.
- The histogram after skewness correction shows a more balanced and symmetric distribution, indicating successful transformation.
- The distribution after standard scaling shows values centered around zero, confirming proper standardization.
- The normalized feature distribution demonstrates that all values are bounded within the range of 0 to 1, making it suitable for algorithms sensitive to scale.

DIMENSIONALITY REDUCTION

```

▶ print("WEEK -07")
#Dimensionality Reduction
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

sns.set_style("whitegrid")

```

... WEEK -07

```

▶ # Data Preparation
df = df.drop(columns=['Unnamed: 0'], errors='ignore')
if 'trans_date_trans_time' in df.columns:
    df['trans_date_trans_time'] = pd.to_datetime(df['trans_date_trans_time'])
    df['hour'] = df['trans_date_trans_time'].dt.hour
    df['day'] = df['trans_date_trans_time'].dt.day
    df['month'] = df['trans_date_trans_time'].dt.month
    df.drop(columns=['trans_date_trans_time'], inplace=True)
else:
    print("Column already removed")

```

```

▶ # Balanced Sampling
fraud = df[df['is_fraud'] == 1]
non_fraud = df[df['is_fraud'] == 0].sample(len(fraud)*5, random_state=42)
df_sample = pd.concat([fraud, non_fraud])
print("Sample Shape:", df_sample.shape)

```

... Sample Shape: (45036, 25)

```
#Numerical Features
numeric_df = df_sample.select_dtypes(include=np.number)
X = numeric_df.drop(columns=['is_fraud'])
y = numeric_df['is_fraud']
```

```
▶ # Feature Selection
selector = SelectKBest(score_func=f_classif, k=8)
X_selected = selector.fit_transform(X, y)
selected_features = X.columns[selector.get_support()]
print("\nSelected Features:", selected_features)
X_selected_df = pd.DataFrame(X_selected, columns=selected_features)

...
Selected Features: Index(['category', 'amt', 'gender', 'dob', 'unix_time', 'amt_scaled', 'hour',
                        'month'],
                        dtype='object')
```

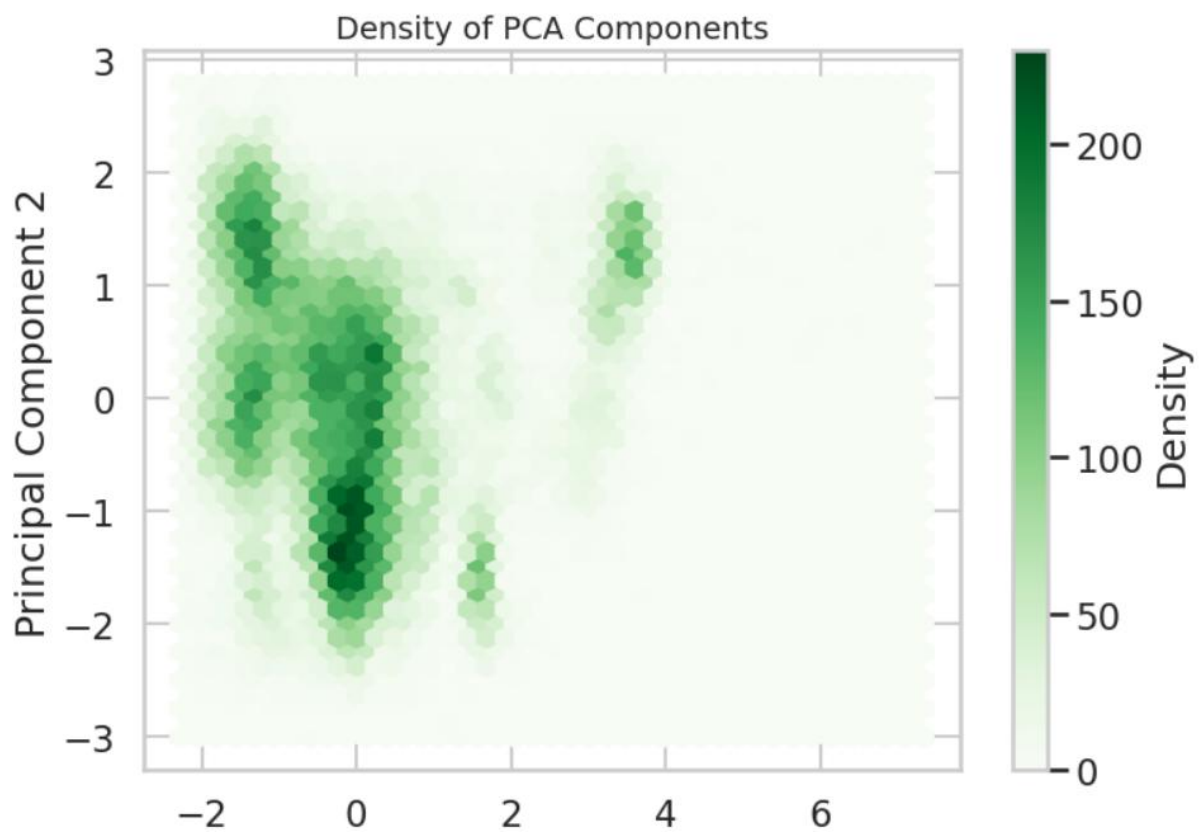
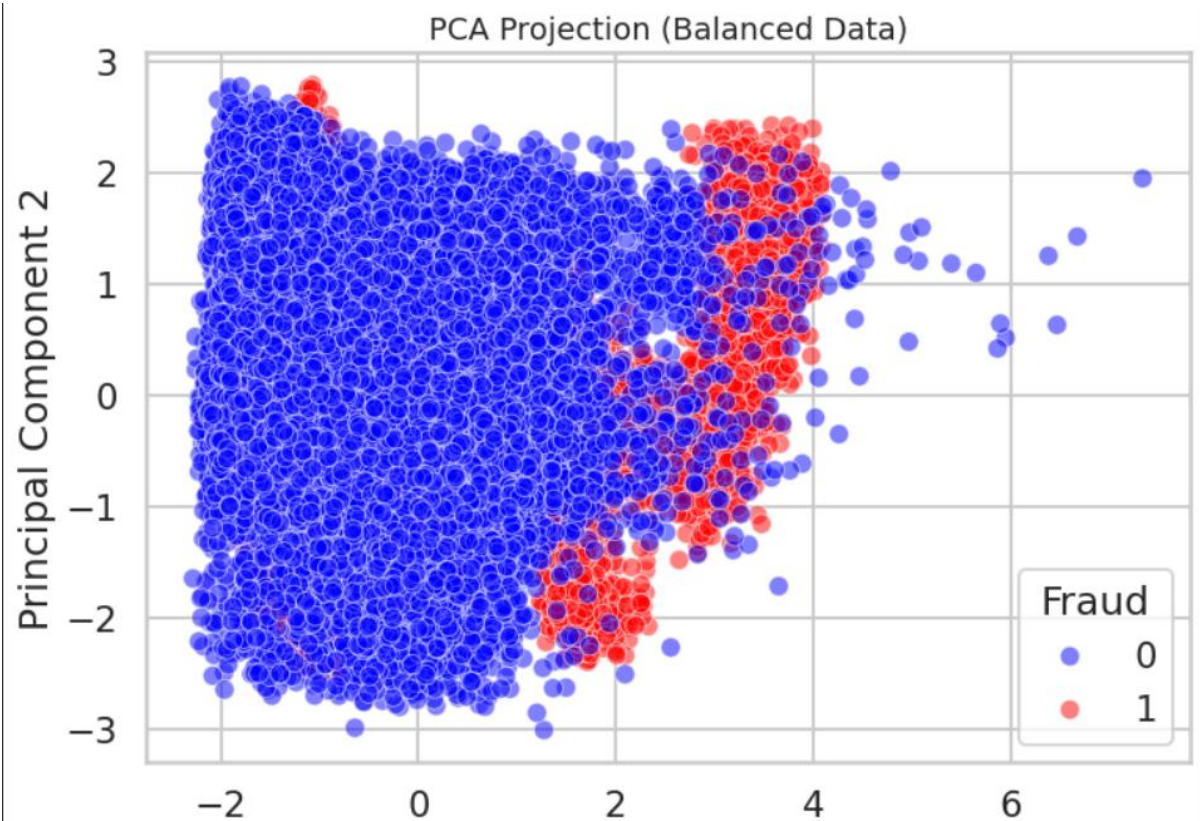
```
# Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_selected_df)
```

```
▶ # PCA (2 Components for Visualization)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

#PCA Scatter Plot
plt.figure(figsize=(8,6))
sns.scatterplot(x=X_pca[:,0], y=X_pca[:,1], hue=y,
               palette={0: "blue", 1: "red"}, alpha=0.5)

plt.title("PCA Projection (Balanced Data)", fontsize=14)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title="Fraud")
plt.tight_layout()
plt.show()

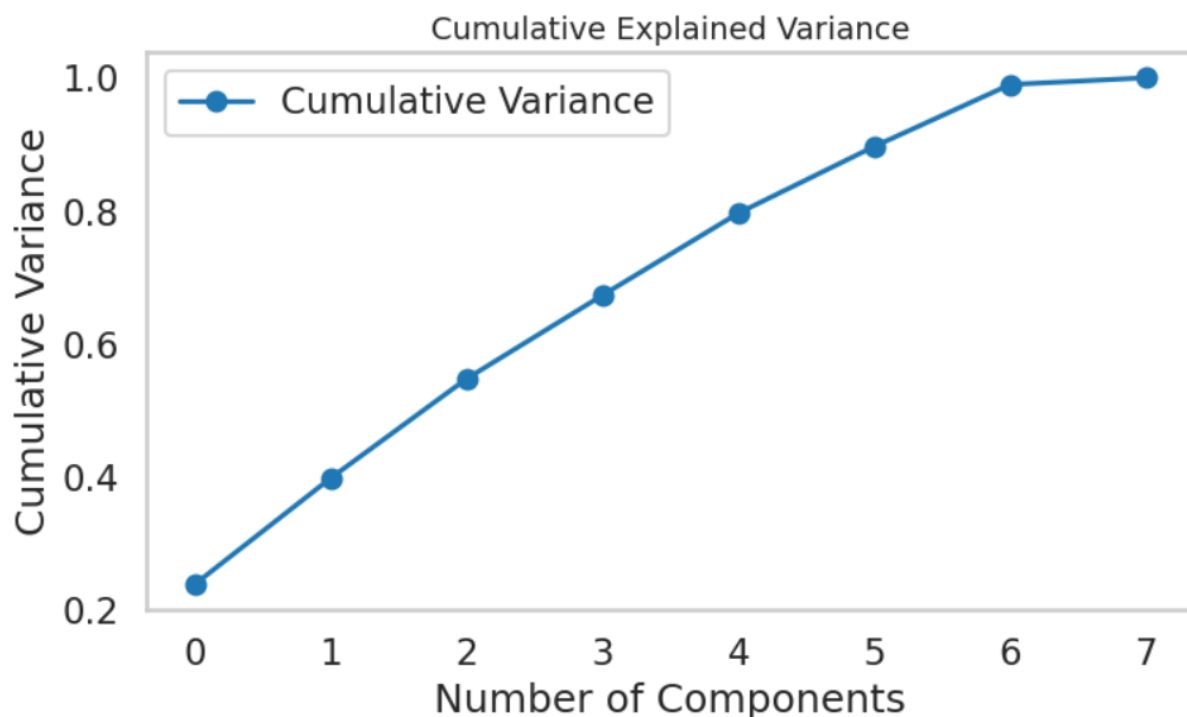
# PCA Density Plot
plt.figure(figsize=(8,6))
plt.hexbin(X_pca[:,0], X_pca[:,1], gridsize=40, cmap='Greens')
plt.title("Density of PCA Components", fontsize=14)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(label="Density")
plt.tight_layout()
plt.show()
```


```

▶ #Explained Variance (Line Plot)
pca_full = PCA()
pca_full.fit(X_scaled)
variance = pca_full.explained_variance_ratio_
plt.figure(figsize=(8,5))
plt.plot(np.cumsum(variance), marker='o', label="Cumulative Variance")
plt.title("Cumulative Explained Variance", fontsize=14)
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Variance")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

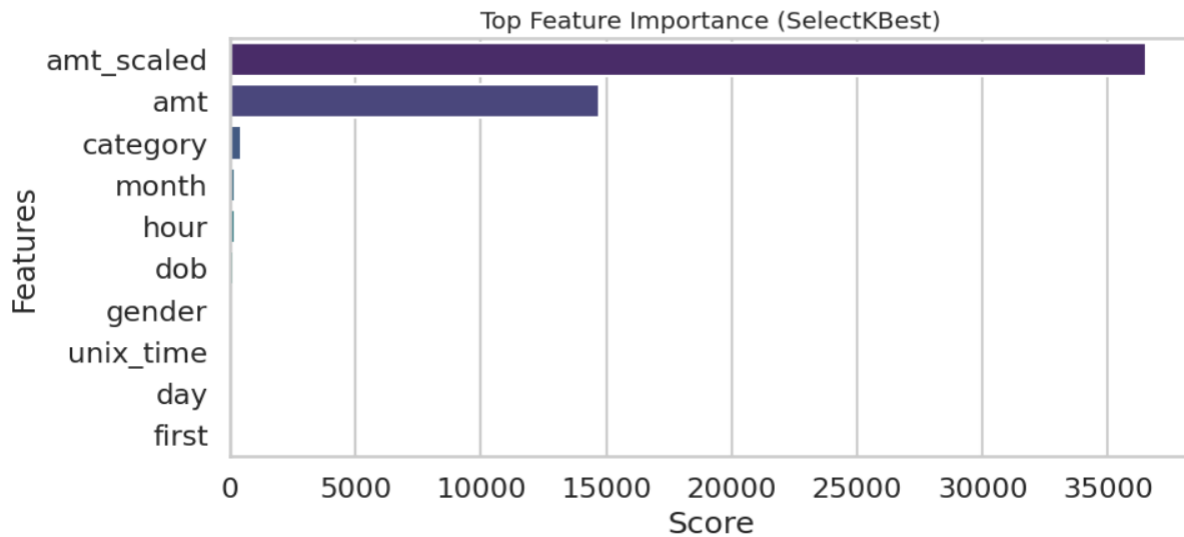
```



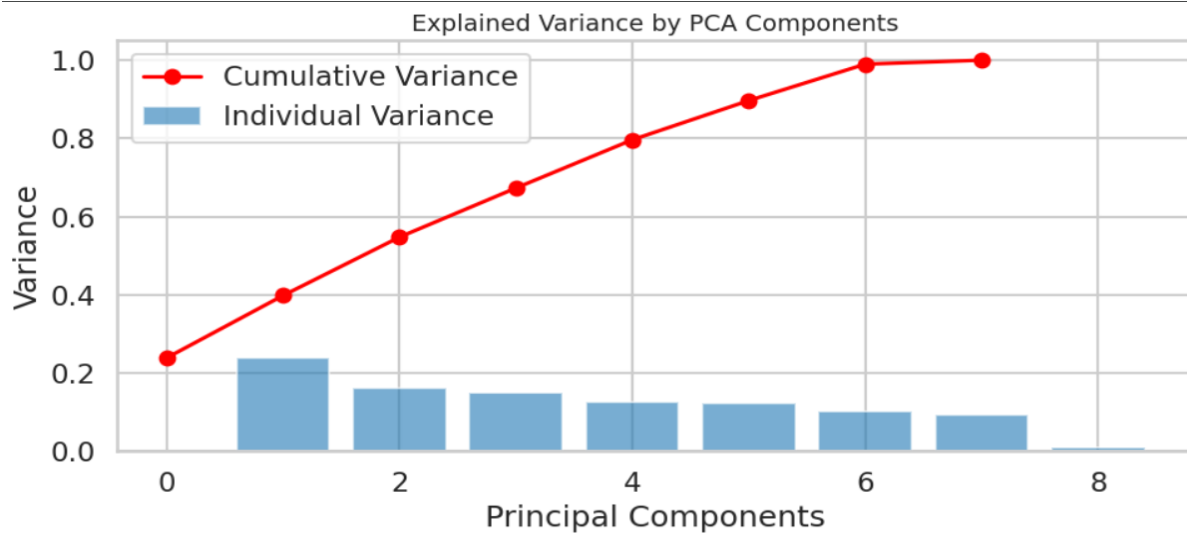
```

▶ #Feature Importance Plot
scores = selector.scores_
feature_scores = pd.DataFrame({
    'Feature': X.columns,
    'Score': scores
}).sort_values(by='Score', ascending=False)
plt.figure(figsize=(10,5))
sns.barplot(x='Score', y='Feature', data=feature_scores.head(10), palette='viridis')
plt.title("Top Feature Importance (SelectKBest)", fontsize=14)
plt.xlabel("Score")
plt.ylabel("Features")
plt.tight_layout()
plt.show()

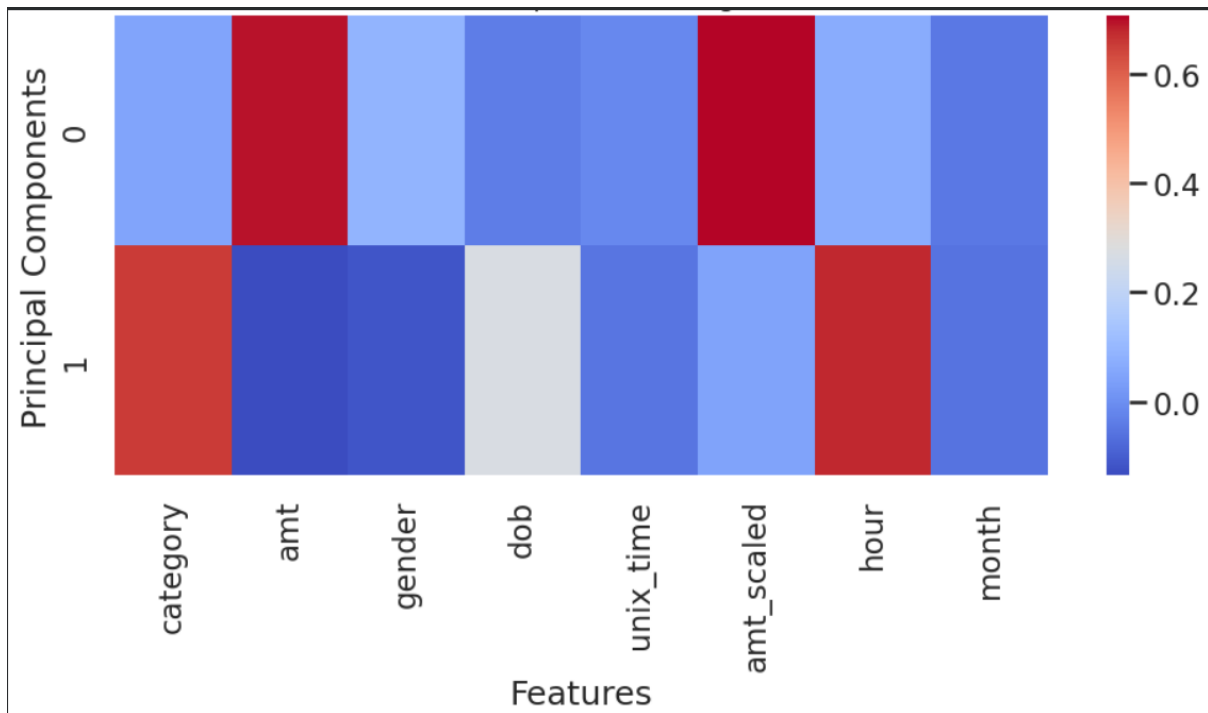
```



```
#Explained Variance (Bar + Line)
plt.figure(figsize=(10,5))
plt.bar(range(1, len(variance)+1), variance, alpha=0.6, label='Individual Variance')
plt.plot(np.cumsum(variance), marker='o', color='red', label='Cumulative Variance')
plt.title("Explained Variance by PCA Components", fontsize=14)
plt.xlabel("Principal Components")
plt.ylabel("Variance")
plt.legend()
plt.tight_layout()
plt.show()
```



```
#PCA Loadings Heatmap
loadings = pd.DataFrame(pca.components_, columns=selected_features)
plt.figure(figsize=(10,6))
sns.heatmap(loadings, cmap='coolwarm')
plt.title("PCA Component Loadings", fontsize=14)
plt.xlabel("Features")
plt.ylabel("Principal Components")
plt.tight_layout()
plt.show()
```



```
# PCA for Better Representation (90% Variance)
print("\nApplying PCA to retain 90% variance")

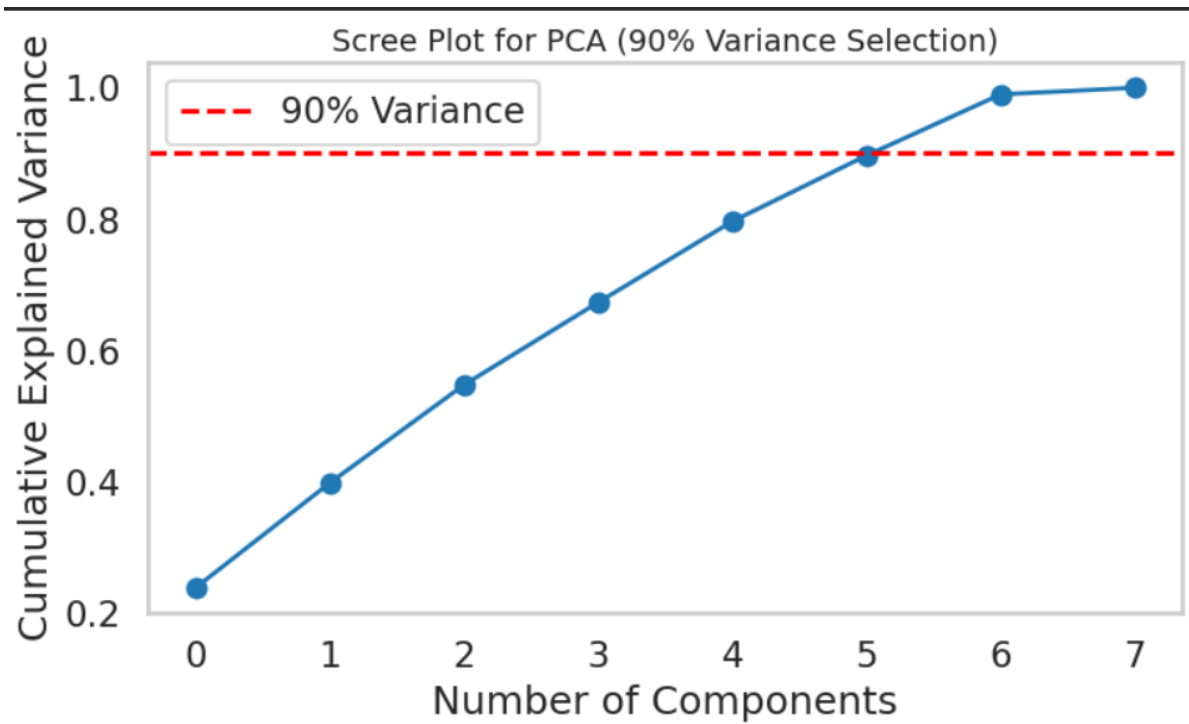
pca_optimal = PCA(n_components=0.90) # retain 90% variance
X_pca_optimal = pca_optimal.fit_transform(X_scaled)

print("Number of Components Selected:", pca_optimal.n_components_)
print("Explained Variance:", np.sum(pca_optimal.explained_variance_ratio_))
```

...

Applying PCA to retain 90% variance
 Number of Components Selected: 7
 Explained Variance: 0.9898858676151947

```
# Scree Plot
plt.figure(figsize=(8,5))
cum_var = np.cumsum(pca_full.explained_variance_ratio_)
plt.plot(cum_var, marker='o', linewidth=2)
plt.axhline(y=0.90, color='red', linestyle='--', label='90% Variance')
plt.title("Scree Plot for PCA (90% Variance Selection)", fontsize=14)
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Explained Variance")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```



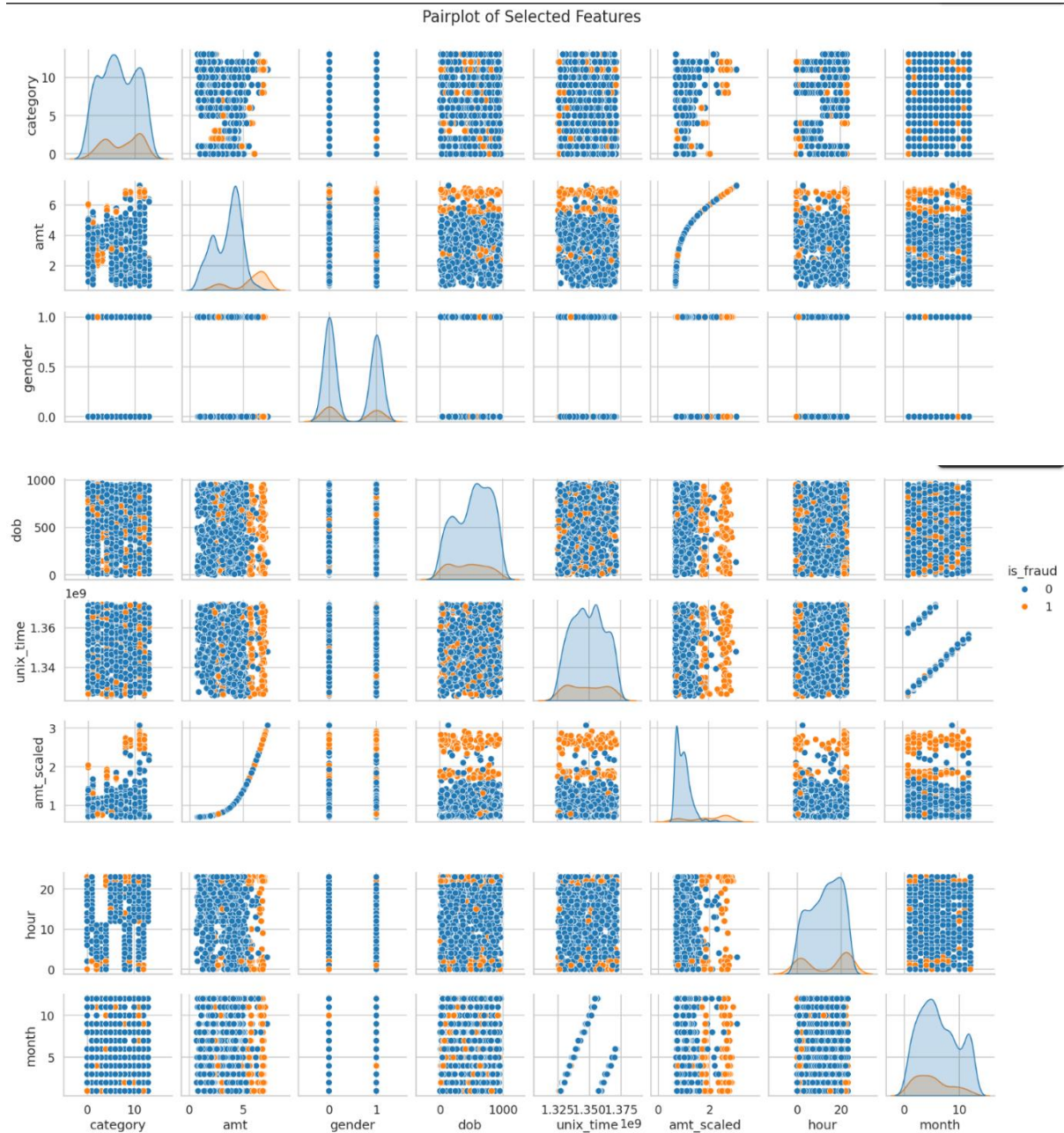
```
# Compare PCA 2 vs PCA 90%
comparison = pd.DataFrame({
    "Method": ["PCA (2 Components)", "PCA (90% Variance)"],
    "Components Used": [2, pca_optimal.n_components_],
    "Variance Retained": [
        np.sum(pca.explained_variance_ratio_),
        np.sum(pca_optimal.explained_variance_ratio_)
    ]
})
print("\nPCA Comparison:\n")
print(comparison)
```

...

PCA Comparison:

	Method	Components Used	Variance Retained
0	PCA (2 Components)	2	0.397702
1	PCA (90% Variance)	7	0.989886

```
#Pairplot (Small Sample)
sample_small = X_selected_df.sample(1000, random_state=42)
sample_small['is_fraud'] = y.sample(1000, random_state=42).values
sns.pairplot(sample_small, hue='is_fraud', diag_kind='kde')
plt.suptitle("Pairplot of Selected Features", y=1.02)
plt.show()
```



INTERPRETATION:

- First, the dataset was prepared by removing unnecessary columns and extracting useful time-based features such as hour, day, and month from the transaction timestamp. This helped in capturing important temporal patterns in the data. A balanced sample was then created by selecting all fraud cases and a proportion of non-fraud cases, making the dataset more suitable for comparison and analysis.

- Next, only numerical features were selected for dimensionality reduction, and the target variable (is_fraud) was separated. Feature selection was performed using the SelectKBest method, which identified the most important features based on statistical significance. This reduced the number of features while retaining the most relevant information.
- After selecting the important features, standard scaling was applied to ensure that all features were on a similar scale. This step is important because dimensionality reduction techniques like PCA are sensitive to differences in feature magnitude.
- Principal Component Analysis (PCA) was then applied to transform the selected features into a smaller set of components. These components capture the maximum variance in the data, allowing the dataset to be represented in fewer dimensions while preserving important patterns.
- Various visualizations such as scatter plots, density plots, and explained variance graphs were used to analyze the transformed data. These plots helped in understanding how well the data is distributed in reduced dimensions and how much information is retained by the principal components.